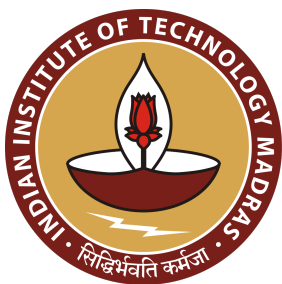


DEPARTMENT OF DATA SCIENCE AND
ARTIFICIAL INTELLIGENCE
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

A Digital-Twin Simulation Framework for Joint Inventory and Transport Optimisation in Multi-Echelon Supply Chains

A Project Report
Submitted by
SAKTHE BALAN

For the award of the degree
Of
INTERDISCIPLINARY DUAL DEGREE PROGRAMME
May 2026



DEPARTMENT OF DATA SCIENCE AND
ARTIFICIAL INTELLIGENCE
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

A Digital-Twin Simulation Framework for Joint Inventory and Transport Optimisation in Multi-Echelon Supply Chains

A Project Report
Submitted by
SAKTHE BALAN

For the award of the degree
Of
INTERDISCIPLINARY DUAL DEGREE PROGRAMME
May 2026

A quotable quote that is quoted by the quoter

– Quote Author

*To whoever, whatever, wherever or whenever, or all of those, or
none at all*

THESIS CERTIFICATE

This is to undertake that the Project Report titled **A DIGITAL-TWIN SIMULATION FRAMEWORK FOR JOINT INVENTORY AND TRANSPORT OPTIMISATION IN MULTI-ECHELON SUPPLY CHAINS**, submitted by me to the Indian Institute of Technology Madras, for the award of **Interdisciplinary Dual Degree Programme**, is a bona fide record of the research work done by me under the supervision of [**Thesis Guide Name**]. The contents of this Project Report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

In order to effectively convey the idea presented in this Thesis, the following work of other authors or sources was reprinted in the Thesis with their permission:

1. Figures ??(a, b, d) and (g) are distributed under the GNU GPL v2 or higher and are free to use. Figures ??(e, f) are distributed under the Creative Commons Attribution-Share Alike 4.0 International license and are free to use
2. Figure ??(c) is copyright Aurelio A. Heckert, and licensed under the Free Art License. Figure ??(h) is freely distributed under the Artistic License v2 or higher

Chennai 600036

Date: May 2026

Sakthe Balan

Dr. Whoever

Research advisor

Professor

Department of Whatever

IIT Madras

Dr. Whoever

Research co-advisor

Professor

Department of Whatever

IIT Madras

LIST OF PUBLICATIONS

I. REFEREED JOURNALS BASED ON THESIS

whatever

II. REFEREED JOURNALS (OTHERS)

whatever

III. PRESENTATIONS IN CONFERENCES

whatever

IV. PUBLICATIONS IN CONFERENCE PROCEEDINGS

whatever

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my thesis guide for their invaluable guidance, encouragement, and support throughout the course of this Interdisciplinary Dual Degree Project. Their insights into supply-chain optimisation and simulation methodology shaped the direction of this work in fundamental ways.

I am grateful to the Department of Data Science and Artificial Intelligence, IIT Madras, for providing the computational resources and academic environment that made this project possible.

I would also like to thank the open-source communities behind SIMPY, OPTUNA, PYTEST, and the broader Python scientific ecosystem, without which this framework could not have been built in the time available.

Finally, I thank my family and friends for their patience and encouragement during the long hours this project demanded.

ABSTRACT

KEYWORDS Multi-echelon inventory optimisation; discrete-event simulation; digital twin; transport consolidation; Bayesian optimisation; TPE; bullwhip effect; supply chain; joint optimisation; Pareto frontier

Classical multi-echelon inventory models optimise stock levels in isolation, treating transport as a linear per-unit cost. In practice, vehicle-based transport has a step-function cost structure: a half-full truck costs nearly the same to operate as a full one, so per-unit transport cost is dictated by vehicle utilisation rather than quantity shipped. When transport accounts for the majority of total supply-chain cost, this mismatch renders inventory-only or transport-only optimisation fundamentally suboptimal.

This project develops a modular, open-source **digital-twin simulation framework** for Multi-Echelon Inventory Optimisation (MEIO) that addresses this gap by co-optimising inventory and transport decisions jointly. The framework implements a discrete-event simulation engine with a daily-tick loop, seven inventory control policies (base-stock, (s, S) , (R, Q) , (R, S) , order-up-to with phase offset, (k, m) -cycle, and echelon base-stock), an arbitrary directed-acyclic-graph network topology, and a transport layer with simultaneous volume and weight capacity constraints. A Tree-structured Parzen Estimator (TPE) optimiser, implemented via Optuna, searches a 20-dimensional mixed-integer space of base-stock levels and dispatch thresholds, subject to a configurable minimum fill-rate constraint.

Experiments are conducted on a 5-SKU, one-supplier–one-warehouse–three-retailer (1N3) network using real demand data from the M5 Walmart Forecasting dataset. The headline result is that joint inventory-transport optimisation (E3) achieves a total cost of \$1,556,516 — a **39% reduction** relative to the analytical Clark-Scarf baseline (\$2,553,418), while simultaneously raising the fill rate from 86.15% to 95.37%, comfortably above the 92% service-level target. The mechanism is a deliberate increase in warehouse safety stock (+\$242k holding cost) that enables high vehicle-consolidation thresholds, reducing transport cost by \$1.04M and yielding a **4.3:1 leverage ratio** between holding and transport savings.

A Pareto frontier analysis across eight fill-rate targets (80%–98%) reveals a natural operating point at approximately 95% fill / \$1.56M and sharply diminishing returns above 94%, providing a principled basis for negotiating service-level agreements. Statistical robustness is confirmed via multi-seed validation with 95% confidence intervals and

Welch's t -tests.

The framework is validated by approximately 80 pytest unit and integration tests and is fully reproducible from a self-contained configuration and demand-data pipeline.

CONTENTS

	Page
ACKNOWLEDGEMENTS	i
ABSTRACT	iii
LIST OF TABLES	ix
LIST OF FIGURES	xi
CHAPTER 1 INTRODUCTION	1
1.1 Multi-Echelon Inventory Optimisation	1
1.2 Why a Digital Twin?	2
1.3 Contributions	3
1.4 Thesis Structure	3
1.5 Development Trajectory	4
CHAPTER 2 LITERATURE REVIEW	7
2.1 Single-Echelon Foundations	7
2.1.1 Newsvendor model	7
2.1.2 Base-stock policy	7
2.1.3 (s, S) and (R, Q) policies	8
2.1.4 Economic Order Quantity (EOQ)	8
2.2 Multi-Echelon Optimality — Clark and Scarf (1960)	8
2.3 Periodic-Review and Cyclic Policies	9
2.3.1 (R, S) periodic-review policy	9
2.3.2 (k, m) -cycle policies	9
2.4 The Bullwhip Effect	10
2.5 Transport Consolidation in Inventory Models	10
2.6 Simulation-Optimisation Methods	11
2.6.1 Genetic algorithms and evolutionary methods	11
2.6.2 Bayesian optimisation and Gaussian processes	11
2.6.3 Tree-structured Parzen Estimator (TPE)	11
2.7 Existing MEIO Software	12
CHAPTER 3 DIGITAL-TWIN ARCHITECTURE	13
3.1 Network Topology	13
3.2 Discrete-Event Simulation Loop	14
3.2.1 Pseudocode	15
3.3 Node State Model	15
3.3.1 Helper methods	16

3.4	Transport Layer	16
3.4.1	TransportOption	17
3.4.2	GreedyLoadPlanner	18
3.4.3	Pluggable protocols	18
3.5	Cost Model	19
3.6	Configuration Schema	20
3.7	CSV Input Pipeline	21
3.8	Supply Disruption Modelling	21
3.9	Iterative Framework Evolution	22
CHAPTER 4 INVENTORY POLICIES		25
4.1	Common API	25
4.2	Base-Stock Policy	26
4.3	Reorder-Point Family: (s, S) and (R, Q)	26
4.3.1	(s, S) policy	26
4.3.2	(R, Q) policy	27
4.4	Periodic-Review: (R, S) and Order-Up-To with Phase Offset	27
4.4.1	(R, S) periodic-review policy	27
4.4.2	Order-up-to with phase offset	28
4.5	Cyclic Policy: (k, m) -Cycle Scheduling	28
4.6	Echelon Base-Stock (Clark–Scarf)	28
4.7	Policy Comparison	29
CHAPTER 5 OPTIMISATION FRAMEWORK		31
5.1	Why Optuna TPE?	31
5.2	Search-Space Encoding — Inventory	32
5.3	Search-Space Encoding — Transport	32
5.4	Joint Search	33
5.5	Constrained Optimisation — Minimum Fill Rate	33
5.6	Pareto Exploration	34
5.7	Re-Evaluation Discipline	34
5.8	Search Space Dimensionality Summary	34
CHAPTER 6 EXPERIMENTAL METHODOLOGY		37
6.1	Preliminary Validation Experiments (EV1–EV4)	37
6.1.1	EV1 — Constant-demand convergence	37
6.1.2	EV2 — Demand shock response	38
6.1.3	EV3 — Policy comparison under transport capacity	38
6.1.4	EV4 — Capacity scaling	38
6.2	Dataset: M5 Walmart Forecasting Data	38
6.3	Main Experiment Configuration	39
6.4	Warm-Up Exclusion	39
6.5	Multi-Seed Statistical Validation	40
6.6	Bullwhip Metric	40
6.7	Main Optimisation Experiments E0–E4	41

6.8	Reproducibility	42
CHAPTER 7	RESULTS AND DISCUSSION	43
7.1	Preliminary Validation Results (EV1–EV4)	43
7.1.1	EV1 — Constant-demand convergence	43
7.1.2	EV2 — Demand shock propagation	43
7.1.3	EV3 — Policy comparison under transport capacity	44
7.1.4	EV4 — Capacity scaling on M5 data	44
7.2	E0 — Analytical Baseline	44
7.3	E1a — Fixed 25% Dispatch Threshold	45
7.4	E1b — Transport-Only Optimisation	46
7.5	E2 — Inventory-Only Optimisation	47
7.6	E3 — Joint Inventory + Transport Optimisation	47
7.6.1	Mechanism of joint optimisation	48
7.6.2	Per-SKU fill rates	49
7.7	E4 — Pareto Frontier: Cost vs Service	49
7.8	Cross-Experiment Comparison	50
7.9	Bullwhip Analysis	51
7.10	Statistical Robustness	51
CHAPTER 8	DISCUSSION	53
8.1	Answering the Research Questions	53
8.1.1	RQ1: Does joint inventory-transport optimisation outperform single-axis optimisation?	53
8.1.2	RQ2: How does the joint-optimised configuration perform under supply disruption?	54
8.1.3	RQ3: Does joint optimisation suppress the bullwhip effect?	54
8.2	Practical Recommendations	55
8.3	Limitations	55
CHAPTER 9	CONCLUSION AND FUTURE WORK	57
9.1	Summary of Contributions	57
9.2	Future Work	58
9.3	Closing Remarks	59
APPENDIX A	CONFIGURATION SCHEMA REFERENCE	61
APPENDIX B	PER-EXPERIMENT POLICY PARAMETERS	65
APPENDIX C	TEST SUITE SUMMARY	67
APPENDIX D	REPRODUCING THE RESULTS	69
BIBLIOGRAPHY		73
CURRICULUM VITAE		75

LIST OF TABLES

Table	Caption	Page
3.1	CSV input files and their roles in the configuration pipeline.	21
4.1	Summary of the seven implemented inventory policies.	30
5.1	Search space dimensionality by optimisation mode (1N3 topology, 5 SKUs).	35
6.1	Preliminary validation experiments: scope and purpose.	37
6.2	Main experiment simulation parameters.	39
6.3	Main optimisation experiments: design and hypothesis.	41
7.1	EV4 capacity scaling results (single-SKU 2W × 3R topology).	44
7.2	E0 cost decomposition and fill rate.	45
7.3	E1a cost decomposition and fill rate.	45
7.4	E1b cost decomposition and fill rate.	46
7.5	E2 cost decomposition and fill rate.	47
7.6	E3 cost decomposition and fill rate (headline result).	48
7.7	Cost composition shift from E0 to E3.	48
7.8	E3 fill rates by SKU.	49
7.9	E4 Pareto frontier: cost vs fill rate under joint optimisation.	49
7.10	Summary of all main optimisation experiments.	50
7.11	Bullwhip ratios (order CV^2 / demand CV^2) under E0 and E3. Ratios > 1 indicate amplification.	51
B.1	E3 optimised base-stock levels.	66
B.2	E3 optimised dispatch thresholds.	66
C.1	Test modules and their coverage areas.	68

LIST OF FIGURES

Figure	Caption	Page
3.1	Example DAG topology: one supplier, two warehouses, and three retailers. The framework supports arbitrary depth and breadth; this is one of several topologies used during development.	14
7.1	Pareto frontier of total cost versus achieved fill rate under joint Optuna-TPE optimisation (E4). Each point represents the best feasible result at the corresponding fill-rate target.	50

CHAPTER 1

INTRODUCTION

1.1 MULTI-ECHELON INVENTORY OPTIMISATION

Inventory management decisions lie at the heart of supply-chain efficiency, resilience, and cost-effectiveness. Modern supply chains are hierarchical: goods flow from raw-material suppliers through distribution centres (DCs) to retail outlets, with inventory held at each stage. Managing inventory across these stages — a problem known as **Multi-Echelon Inventory Optimisation (MEIO)** — requires making critical replenishment decisions under demand uncertainty, transportation limitations, and operational constraints, often without clear visibility into system-wide consequences.

Three compounding difficulties make the MEIO problem harder than its single-echelon counterpart:

1. **Coupling of stockouts.** A stockout at a DC propagates downstream: retailers awaiting replenishment accumulate unfilled demand (backlog), incurring shortage costs. Simultaneously, the DC's depleted inventory triggers a large replenishment order upstream, stressing the supplier's capacity.
2. **Lead-time amplification.** The effective replenishment lead time seen by a retailer is the cumulative sum of transit times across all upstream echelons. With a 2-day supplier-to-DC lead time and a 1-day DC-to-retailer lead time, the retailer's effective lead time is 3 days — and any safety stock calculation must account for demand variability over the entire 3-day window, not just the final mile.
3. **The bullwhip effect.** Small fluctuations in end-customer demand are amplified as orders propagate upstream. A retailer reacting to a demand spike places a large order at the DC; the DC, uncertain of future demand, orders even more from the supplier. This amplification can cause the supplier to swing between feast and famine, eroding efficiency across the chain (Lee *et al.*, 1997).

These interdependencies mean that optimising inventory at each node independently — the standard single-echelon approach — is suboptimal or even infeasible. A system-level view is required.

Compounding this challenge, modifying inventory policies in a live supply chain carries significant operational risk. Downstream impacts on service levels, costs, and system stability are difficult to anticipate in advance: a change to replenishment parameters at the warehouse level may look locally optimal yet trigger bullwhip-amplified order swings at the supplier, or deplete retailer safety stock to dangerous levels several weeks later. This makes experimentation on production systems costly and often impractical.

1.2 WHY A DIGITAL TWIN?

The classical solution to the MEIO problem is the Clark-Scarf echelon base-stock policy (Clark and Scarf, 1960), which is optimal for serial networks under a set of idealising assumptions:

- Backlog symmetry: unmet demand is fully backordered at a known linear penalty (no lost sales, no emergency orders).
- Infinite supply at the upstream end: the supplier can always meet any order placed.
- Deterministic lead times: every shipment arrives in exactly L days.
- No transport capacity: vehicles can carry any quantity at the same per-unit cost.

Real supply chains routinely violate all four assumptions. Suppliers have capacity limits; lead times vary with congestion; transport costs are non-linear functions of vehicle utilisation. When transport cost dominates the budget — as in the system studied in this DDP, where it accounts for 74% of total cost at the analytical baseline — ignoring vehicle capacity and consolidation leaves enormous savings on the table.

In practice, decision-makers frequently rely on simulation tools to evaluate inventory policies and operational scenarios before deployment in order to avoid the operational risks described above. However, many available simulation tools are proprietary, domain-specific, or tightly coupled to particular industrial settings, limiting their transparency, extensibility, and suitability for systematic research (Chapter 2 surveys the landscape in detail). An open, configurable, MEIO-specific simulation framework that handles

multi-SKU demand, arbitrary network topologies, and vehicle-capacity-aware transport has not previously been available.

A **digital-twin simulation** is a computational replica of a supply chain that can evaluate *any* policy under *any* set of assumptions. Unlike closed-form models, it does not need restrictive assumptions to be tractable. Unlike real-world pilots, it can evaluate thousands of policy configurations in hours rather than months, and unlike proprietary tools, it can be inspected, extended, and adapted to new settings. The simulation-optimisation paradigm — using a fast simulator as the inner loop of a black-box optimiser — has emerged as the practical approach for supply-chain design when analytical solutions are unavailable or inapplicable (Fu *et al.*, 2005).

1.3 CONTRIBUTIONS

This DDP makes the following research contributions:

1. **A modular, tested digital-twin simulation framework for MEIO.** The framework implements seven inventory control policies (base-stock, (s, S) , (R, Q) , (R, S) , order-up-to with phase offset, (k, m) -cycle, and echelon base-stock), an arbitrary DAG-topology network model, and a transport layer that enforces simultaneous volume and weight capacity constraints with tiered cost schedules. The codebase is validated by approximately 80 pytest tests covering invariants, policy behaviour, and transport mechanics.
2. **An Optuna-TPE-based joint optimiser with fill-rate constraints.** The optimiser searches over a mixed-integer space of up to 20 parameters (15 base-stock levels + 5 dispatch thresholds) and enforces a minimum fill-rate target via a soft penalty. Three modes are supported: inventory-only, transport-only, and joint.
3. **Empirical demonstration that joint optimisation outperforms single-axis optimisation.** On a 5-SKU, 1N3 network with M5 Walmart demand, joint optimisation achieves a 39% total cost reduction relative to the analytical newsvendor baseline while simultaneously raising fill rate from 86% to 95% — results that neither inventory-only nor transport-only optimisation can match.

1.4 THESIS STRUCTURE

The remainder of this thesis is organised as follows.

Chapter 2 reviews the theoretical foundations: single-echelon newsvendor theory, Clark-Scarf multi-echelon optimality, periodic-review policies, the bullwhip effect, transport consolidation models, and simulation-optimisation methods.

Chapter 3 describes the digital-twin framework architecture: the DAG network model, the discrete-event simulation loop, the node state machine, the transport layer, the cost model, and the JSON and CSV configuration pipelines.

Chapter 4 presents the seven inventory policies, their mathematical formulations, and a comparative discussion of their strengths and limitations.

Chapter 5 describes the Optuna-TPE optimisation framework: search-space encoding, the fill-rate constraint mechanism, and the re-evaluation discipline.

Chapter 6 details the experimental design, dataset (M5 Walmart), warm-up exclusion procedure, multi-seed statistical validation, and the six main optimisation experiments (E0–E4).

Chapter 7 presents the results of all experiments with full cost decompositions, fill-rate breakdowns, bullwhip ratios, and a cross-experiment comparison.

Chapter 8 synthesises the results, answers the research questions, draws practical recommendations, and acknowledges limitations.

Chapter 9 summarises the contributions, quantifies the headline results, and outlines the future research roadmap.

1.5 DEVELOPMENT TRAJECTORY

The DDP unfolded in three phases over approximately 1.5 years.

Phase 1 — Prototype and validation. A single-SKU simulator was built on a simplified

2-warehouse $\times$ 3-retailer topology. Four controlled validation experiments (EV1–EV4) using synthetic and M5 Walmart demand confirmed that the discrete-event loop, shipment pipeline, and backlog mechanics behave as predicted by theory. This phase established confidence in the simulator’s correctness before adding complexity.

Phase 2 — Transport layer and scale-up. The most significant architectural addition in this phase was the **transport module**: a vehicle-capacity-aware layer implementing dual volume and weight constraints, tiered cost schedules (full-, half-, and quarter-load rates), and a configurable minimum dispatch utilisation threshold with a maximum-wait timeout. This module transformed the simulator from a pure inventory tool into a joint inventory-transport digital twin — the distinguishing feature of the framework. Concurrently, the network was extended to the 5-SKU 1N3 topology, the inventory policy suite grew from two implementations (base-stock, (s, S)) to seven, and a comprehensive test suite, multi-seed validation, and bullwhip metric were instrumented.

Phase 3 — Joint optimisation. The Optuna-TPE optimiser was integrated and the main experiments (E0–E4) were designed and executed. The joint optimiser searches inventory base-stock levels and transport dispatch thresholds simultaneously — a search structure only made meaningful by the transport layer built in Phase 2. Joint optimisation of inventory and transport parameters emerged as the central research contribution, achieving a 39% cost reduction over the analytical baseline while exceeding the 92% fill-rate target.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews the theoretical foundations underlying the DDP and situates the contribution within the existing literature. Seven areas are covered: single-echelon inventory theory, Clark-Scarf multi-echelon optimality, periodic-review and cyclic policies, the bullwhip effect, transport consolidation, simulation-optimisation methods, and existing open-source MEIO software.

2.1 SINGLE-ECHELON FOUNDATIONS

2.1.1 Newsvendor model

The **newsvendor model** (Arrow *et al.*, 1951) is the canonical single-period stocking problem. A retailer must commit to a stock level S before random demand D is realised; unsold units cost h per unit (holding) while unmet demand costs p per unit (shortage). The optimal stock level equates the critical ratio:

$$F(S^*) = \frac{p}{p + h} \quad (2.1)$$

where F is the CDF of demand. For Gaussian demand this yields the service-level formula $S^* = \mu + z_\alpha \sigma$, where z_α is the quantile of the standard normal corresponding to the target fill rate α .

2.1.2 Base-stock policy

The **base-stock (order-up-to) policy** extends the newsvendor to a multi-period setting with backlogging. Each period the stocking point orders up to a fixed target S , which is also the *inventory position* target. Under i.i.d. demand and deterministic lead time L , the optimal base-stock level is:

$$S^* = \mu(L + 1) + z_\alpha \sigma \sqrt{L + 1} \quad (2.2)$$

This formula is used to initialise the analytical baseline in experiment E0 (Section 7.2).

2.1.3 (s, S) and (R, Q) policies

The (s, S) **policy** places an order to restore inventory to level S whenever the inventory position falls below s (Scarf, 1960). It is optimal for single-echelon problems with fixed ordering costs under mild conditions. The (R, Q) **policy** places a fixed quantity Q whenever the inventory position reaches the reorder point R , which aligns naturally with discrete vehicle load sizes.

2.1.4 Economic Order Quantity (EOQ)

The **EOQ model** (Harris, 1913) minimises the sum of ordering and holding costs in a deterministic, continuous-review setting:

$$Q^* = \sqrt{\frac{2Kd}{h}} \quad (2.3)$$

where K is the fixed cost per order, d is the demand rate, and h is the holding cost rate. EOQ is a useful reference for calibrating order quantities in the (R, Q) policy but ignores service levels and capacity constraints.

2.2 MULTI-ECHELON OPTIMALITY — CLARK AND SCARF (1960)

Clark and Scarf (1960) established the foundation of MEIO by proving that, for a serial network with backlogging, independent demands, and infinite supplier capacity, the **echelon base-stock policy** is optimal.

The key insight is the *echelon inventory position* (EIP): rather than conditioning the replenishment decision at echelon i on only the local on-hand and in-transit stock, the

policy conditions on the stock at echelon i *plus* all downstream echelons. This accounts for the fact that inventory at echelon i will eventually flow downstream and must satisfy downstream demand as well as local demand.

The Clark-Scarf decomposition reduces the multi-echelon problem to a sequence of single-echelon newsvendor problems, one per echelon, solved in downstream-to-upstream order. The solution is optimal under the following assumptions:

- Serial (chain) topology — no divergent DCs serving multiple retailers.
- Stationary, independent demands at each echelon.
- No capacity constraints on orders or vehicles.
- Backlogging (no lost sales).

When these assumptions are violated — as they are in the divergent 1N3 topology with vehicle capacity and real M5 demand — the Clark-Scarf solution is no longer guaranteed to be optimal. This gap motivates the simulation-optimisation approach taken in this DDP.

2.3 PERIODIC-REVIEW AND CYCLIC POLICIES

2.3.1 (R, S) periodic-review policy

The (R, S) **policy** reviews inventory every R periods and orders up to level S . It reduces the administrative overhead of continuous-review policies and is widely used in practice. Optimal values of R and S can be found by exhaustive search or approximation (Silver, 1978).

2.3.2 (k, m) -cycle policies

Roundy (1985) showed that (k, m) -**cycle policies** — which allow at most k orders within an m -period cycle — can achieve near-optimal cost in multi-item inventory systems while enabling *co-ordinated replenishment*. By fixing the days on which each item is reviewed, transport planners can schedule full-truckload deliveries in advance, reducing the empty-vehicle problem that plagues unco-ordinated ordering.

2.4 THE BULLWHIP EFFECT

Lee *et al.* (1997) identified and formalised the **bullwhip effect**: the tendency for demand variability to amplify as orders propagate upstream in a supply chain. They attributed bullwhip to four root causes: demand signal processing (over-reaction to demand fluctuations), order batching, price fluctuations, and rationing games.

The most widely used empirical metric for bullwhip is the ratio of order variability to demand variability (Chen *et al.*, 2000):

$$\text{Bullwhip ratio} = \frac{\text{CV}^2(\text{orders placed by node } n)}{\text{CV}^2(\text{demand faced by node } n)} \quad (2.4)$$

A ratio greater than one at an upstream node indicates amplification. This metric is computed for all nodes and SKUs in the post-warm-up window of each simulation run (Section 6.6).

2.5 TRANSPORT CONSOLIDATION IN INVENTORY MODELS

Classical MEIO models treat transport cost as a linear function of quantity shipped, ignoring the step-function cost structure of vehicle-based transport. In practice, a truck that is 50% full and a truck that is 100% full cost nearly the same to operate; the cost-per-unit is therefore roughly twice as high for the half-full truck.

Jagerman *et al.* (1990) and subsequent work on *inventory with vehicle routing* showed that the interaction between inventory and transport decisions can be significant: raising safety stock at the retailer allows the supplier to batch orders and dispatch full trucks, reducing per-unit transport cost by 30–60% in realistic settings.

Despite this, the dominant MEIO literature — including Clark-Scarf, Graves-Willems, and most textbook treatments — ignores vehicle capacity and consolidation. This creates the gap that the joint optimisation framework in this DDP addresses: when transport cost

dominates the budget, *inventory and transport decisions must be optimised jointly*.

2.6 SIMULATION-OPTIMISATION METHODS

When the performance surface of a stochastic or complex system cannot be expressed analytically, **simulation-optimisation** uses a simulator as an oracle and an optimisation algorithm to search the parameter space (Fu *et al.*, 2005).

2.6.1 Genetic algorithms and evolutionary methods

Genetic algorithms (GAs) have been applied to MEIO parameter tuning since the 1990s. They are flexible and parallelisable but typically require thousands of function evaluations to converge and are not sample-efficient for smooth, low-dimensional surfaces.

2.6.2 Bayesian optimisation and Gaussian processes

Bayesian optimisation fits a probabilistic surrogate (typically a Gaussian process) to observed function values and uses an acquisition function (e.g. expected improvement) to choose the next evaluation point. It is highly sample-efficient for smooth functions in up to ~ 20 dimensions but scales cubically in the number of observations, making it slow for larger search spaces.

2.6.3 Tree-structured Parzen Estimator (TPE)

The **TPE algorithm** (Bergstra *et al.*, 2011) replaces the Gaussian-process surrogate with two separate kernel density estimators: one for the distribution of parameters that led to good results ($\ell(x)$) and one for the rest ($g(x)$). The acquisition function is $\ell(x)/g(x)$, maximised by drawing many samples from $\ell(x)$ and selecting the one with the highest ratio.

TPE is well-suited to this DDP's optimisation problem because:

- It handles mixed-integer variables (integer base-stock levels and continuous dispatch thresholds) natively.
- It scales approximately linearly in dimensionality, remaining tractable for the 20-dimensional joint search space.

- It is implemented in **OPTUNA** (Akiba *et al.*, 2019), an open-source framework with support for parallelism and constraint handling.

2.7 EXISTING MEIO SOFTWARE

Several simulation and optimisation tools are available for supply-chain modelling, but none satisfies all requirements of this DDP simultaneously.

SIMPY A Python discrete-event simulation library. General-purpose and well-maintained, but provides no MEIO domain primitives (no nodes, no policies, no transport). Building an MEIO simulator from SimPy requires substantial boilerplate.

ANYLOGIC A commercial simulation platform with a supply-chain library. Powerful and well-documented, but proprietary, expensive, and not scriptable for large-scale parameter sweeps.

SIMOPT An open-source Python library for simulation-optimisation (Pasupathy and Poloczek, 2022). Provides a collection of problem instances and solvers, but does not include MEIO-specific features such as multi-SKU demand, vehicle capacity, or transport consolidation.

OPTUNA An open-source Bayesian/TPE optimisation framework (Akiba *et al.*, 2019). Used as the outer optimisation loop in this DDP; does not include simulation functionality.

The gap identified is clear: there is no open, MEIO-specific simulation + optimisation framework that handles multi-SKU demand, arbitrary DAG topologies, and vehicle-capacity-aware transport. This DDP fills that gap.

CHAPTER 3

DIGITAL-TWIN ARCHITECTURE

This chapter describes the design and implementation of the discrete-event simulation framework that forms the computational core of the DDP. Each section maps to one or more modules in the `engine/` package and explains the algorithmic choices made at that layer.

3.1 NETWORK TOPOLOGY

The framework represents any supply-chain network as a **directed acyclic graph (DAG)**, implemented in `engine/network.py`. Each *node* in the graph corresponds to a stocking location (supplier, warehouse, or retailer) and each directed *edge* to a transport lane between two locations. Goods flow strictly downstream; returns and lateral transshipment are not modelled.

The topology is entirely configurable: the number of echelons (chain depth) and the number of nodes at each echelon (chain breadth) are controlled by the JSON configuration file without any change to simulator code. A single supplier feeding one warehouse and three retailers, a two-DC hub-and-spoke network, or a flat one-level distribution system are all expressible as DAG configurations. Throughout this work, several topologies of varying size and depth were used; the specific configuration for each experiment is described in Chapter 6.

Two further design choices are worth noting. First, the supplier node carries an optional `infinite_supply` flag: when set, the supplier can always fulfil any order placed by downstream nodes, representing an ideally reliable upstream source. This assumption is convenient for isolating the behaviour of downstream inventory policies but

is entirely optional — finite-capacity suppliers and supply disruptions are also supported (Section 3.8). Second, the DAG structure enforces a strict material-flow direction, ensuring that cycle detection is trivial and the topological processing order (Section 3.2) is unique.

*[Figure: Example multi-echelon supply network —
Supplier → Warehouses W1, W2 → Retailers R1, R2, R3.
Arrows show downstream material flow; lead times
labelled on each lane.]*

Figure 3.1: Example DAG topology: one supplier, two warehouses, and three retailers. The framework supports arbitrary depth and breadth; this is one of several topologies used during development.

3.2 DISCRETE-EVENT SIMULATION LOOP

The simulator advances time in unit steps of one day. Each day t executes the following sequence of operations:

1. **Receive shipments.** For every node, any shipment whose scheduled arrival time equals t is removed from the pipeline and added to on-hand inventory.
2. **Realise demand.** Each retailer node draws its external demand for day t from the configured demand generator (deterministic, Poisson, or CSV-driven).
3. **Clear external backlog.** Any accumulated external demand that could not be served on previous days is fulfilled from current on-hand stock, up to availability.
4. **Process child orders.** Internal orders from downstream nodes are served from the parent’s on-hand inventory; unmet orders enter the parent’s *children backlog*.
5. **Place replenishment orders.** Each non-supplier node queries its inventory policy to determine an order quantity and registers an `IncomingOrder` at its parent.
6. **Dispatch planning.** For each edge, the transport module inspects accumulated orders, builds vehicle loads respecting capacity constraints, and schedules shipments with the configured lead time (Section 3.4).
7. **Cost accumulation.** Holding, shortage, ordering, and transport costs for day t are recorded.

Nodes are processed in *topological order* — retailers before warehouses, warehouse

before the supplier — so that each node's child-order step is complete before its parent's dispatch step. The topological sort is computed once at simulation startup (`Simulator._topological_order`).

The entry point is `Simulator.run(mode)`, implemented in `engine/simulator.py`. When `mode="logs"` the simulator records full daily time-series; when `mode="summary"` only the aggregate KPI row is returned, which is sufficient for the optimisation inner loop.

3.2.1 Pseudocode

Algorithm 1 summarises one daily tick.

Algorithm 1 Daily simulation tick at time t	
1:	for each node n in topological order do
2:	$n.receive_shipments(t)$
3:	if n is a retailer then
4:	demand $\leftarrow$ demandGenerator[n].get(t)
5:	$n.process_external_demand(demand, t)$
6:	end if
7:	$n.clear_external_backlog()$
8:	$n.process_child_orders(t)$
9:	end for
10:	for each non-supplier node n in topological order do
11:	$q \leftarrow n.policy.order_qty(on_hand, backlog, pipeline_in, t)$ $\triangleright q$: order quantity in units
12:	$n.parent.add_inbound_order(n, q, t)$
13:	end for
14:	for each edge e in the network do
15:	loads $\leftarrow$ GreedyLoadPlanner.plan($e.pending_orders$)
16:	for each load ℓ in loads do
17:	schedule shipment arrival at $t + e.lead_time$
18:	accumulateCost(e, ℓ)
19:	end for
20:	end for
21:	accumulateCost(t) $\triangleright$ holding and shortage

3.3 NODE STATE MODEL

Each node in the network maintains the following per-SKU state variables, implemented as dictionaries keyed by SKU identifier in `engine/node.py`:

on_hand Current physical stock available for dispatch or external fulfilment.

pipeline_in A list of `Shipment` objects, each carrying a quantity and a scheduled arrival day. The total pipeline quantity is the sum of all in-transit units.

backlog_external Unfulfilled external demand accumulated over previous days; cleared before new demand is processed on each tick.

backlog_children A dictionary keyed by `(child_id, sku)` recording how many units the parent owes to each downstream child.

inbound_orders_queue Orders submitted by children that have not yet been dispatched.

placed_orders Historical record of orders placed by this node; used to compute the bullwhip ratio.

The two backlog dictionaries encode the dual role of intermediate nodes: they both *sell* to their children (`backlog_children`) and *buy* from their parent (`backlog_external` records their own unmet need).

3.3.1 Helper methods

`Node.total_pipeline_in(sku)` returns the scalar sum over all in-transit shipments for a given SKU, which is the quantity passed to the inventory policy as `pipeline_in`.

`Node.total_backlog_children(sku)` aggregates children-owed quantities across all child nodes, used by the echelon-stock calculation (Section 4.6).

3.4 TRANSPORT LAYER

Transport between nodes is modelled at the *vehicle load* level rather than at the individual unit level. The transport layer is implemented in `engine/transport.py` and consists of three collaborating types.

A key design feature is that a single lane (edge) between two nodes can carry **multiple transport options** simultaneously — one per available mode. For example, a warehouse-to-retailer lane might offer both a truck option (high capacity, 1-day lead time) and a rail option (higher capacity, 3-day lead time, lower cost per vehicle). This directly mirrors real-world logistics, where two locations are often served by more than one carrier type, each with its own cost structure and transit time. Multiple options are declared by listing several edge entries with the same `from/to` pair in the config's `edges` array, each with a distinct `mode` integer.

3.4.1 TransportOption

A `TransportOption` specifies the physical and economic properties of one transport mode on a lane:

- **mode** — a string label (e.g. "truck", "rail").
- **capacity** — maximum volume (in cubic metres) per vehicle.
- **weight_capacity** — maximum gross weight (kg) per vehicle; `None` means no weight constraint.
- **cost_full / cost_half / cost_quarter** — tiered cost schedule: a full load costs `cost_full`, a load between 50% and 100% utilisation costs `cost_half`, and a load below 50% costs `cost_quarter`.
- **lead_time** — deterministic transit days (stochastic lead times are a planned extension).
- **min_dispatch_utilization** — minimum fraction of capacity that must be filled before a vehicle is released. A value of 0 means dispatch on any positive load; a value of 0.8 means the planner waits until 80% of capacity is committed.

When multiple `TransportOptions` exist on a lane, the framework's default **mode selector** sorts them by their `mode` integer (1 = primary, 2 = secondary, ...) and dispatches all cargo via the lowest-numbered option, with higher-numbered modes serving as fallbacks. In addition, the optional `share` field on an edge enables *weighted random sampling* of lead times across options — for example, assigning `share: 0.8` to a truck route and `share: 0.2` to a rail route means 80% of dispatch events use

the truck lead time and 20% use the rail lead time, directly modelling realistic carrier variability for multi-route logistics corridors without splitting individual vehicle loads.

3.4.2 GreedyLoadPlanner

The GreedyLoadPlanner is the load-packing algorithm used in all experiments in this work. It is the default implementation of the LoadPlanner protocol. Given a list of pending order lines for a lane, it greedily packs items into vehicles one at a time, subject to dual constraints:

1. **Volume constraint:** total volume of packed items $\leq$ capacity.
2. **Weight constraint:** total weight of packed items $\leq$ weight_capacity (when set).

If weight capacity is tighter than volume capacity for the current mix of SKUs, the effective capacity is scaled down accordingly before packing begins. After packing, if the resulting utilisation is below `min_dispatch_utilization`, the load is *held* for up to `max_dispatch_wait` days; once the wait limit is reached the load is dispatched regardless of fill level to prevent indefinite stockouts at downstream nodes.

The tiered cost is evaluated after packing: loads $\geq 50\%$ full pay the half-load rate, loads $\geq 100\%$ (i.e. exactly full) pay the full-load rate. This incentivises consolidation but never blocks dispatch beyond the wait limit.

3.4.3 Pluggable protocols

The transport layer is designed around three narrow protocols, making all three components fully **plug-and-play**:

LoadPlanner Decides how pending orders are packed into vehicle loads.

GreedyLoadPlanner is the default used in this work; alternatives (bin-packing, probabilistic packing) can be substituted by implementing the same two-method interface.

ModeSelector Chooses which transport option(s) to activate when multiple options

exist on a lane. The default `CheapestModeSelector` always picks the primary mode (lowest mode integer); a custom selector could split loads across modes or apply routing rules.

CostAllocator Distributes a vehicle's total cost across the SKUs it carries. The default allocates cost proportionally by volume. No changes to the simulator core (`engine/simulator.py`) are required to swap any of these components — they are injected at construction time, making the transport layer independently extensible.

3.5 COST MODEL

The total cost accumulated over the simulation horizon T is:

$$C_{\text{total}} = C_{\text{hold}} + C_{\text{transport}} + C_{\text{order}} + C_{\text{shortage}} \quad (3.1)$$

Each component is defined as follows.

Holding cost. $C_{\text{hold}} = \sum_{t=w}^T \sum_n \sum_s h_{n,s} \cdot \text{on_hand}_{n,s,t}$, where $h_{n,s}$ is the per-unit-per-day holding cost for SKU s at node n and w is the warm-up cutoff.

Shortage (backlog) cost. $C_{\text{shortage}} = \sum_{t=w}^T \sum_n \sum_s p_{n,s} \cdot \text{backlog_external}_{n,s,t}$, where $p_{n,s}$ is the per-unit-per-day penalty cost.

Ordering cost. Each order placed by a node incurs a fixed setup cost K_n plus a per-unit variable cost $c_{n,s}$: $C_{\text{order}} = \sum_{\text{orders}} (K_n + c_{n,s} \cdot q)$, where q is the order quantity in units.

Transport cost. Each dispatched vehicle load on edge e with utilisation u incurs a tiered cost: `cost_full` if $u = 1$, `cost_half` if $0.5 \leq u < 1$, and `cost_quarter` if $u < 0.5$.

All four components are accumulated on every tick inside `Simulator.run` and the first w days are excluded from reported KPIs to avoid contamination by the initial-inventory transient (Section 6.4).

3.6 CONFIGURATION SCHEMA

Scenarios are specified as JSON files. The top-level keys are:

seed Integer random seed for reproducibility.

time_horizon Number of simulation days.

skus List of SKU identifiers.

sku_properties Per-SKU `volume_per_unit` and `weight_per_unit`.

nodes List of node objects specifying type (`supplier/warehouse/retailer`), initial inventory, cost parameters, and per-SKU policy configuration.

edges List of edge objects specifying from/to nodes and a list of `TransportOption` specifications.

disruptions (Optional) List of supply disruption events; each event specifies a node, a start day, and a duration.

The `engine/config_validator.py` module validates all required fields and raises descriptive errors before the simulation starts, preventing silent misconfiguration during long experiment runs.

Randomness in the framework arises from two sources: (i) stochastic demand generators — for example, Poisson-distributed daily demand draws a fresh random variate each day — and (ii) the Optuna TPE optimiser, which uses probabilistic surrogate models and random sampling to explore the parameter space. The seed value initialises Python’s random module and NumPy’s RNG at simulation startup, ensuring that both demand sequences and optimiser trial sequences are exactly reproducible across runs. CSV-driven demand (used in all main experiments of this work) is fully deterministic and unaffected by the seed; the seed matters primarily when using Poisson or other synthetic demand modes and when running the optimiser.

3.7 CSV INPUT PIPELINE

To lower the barrier for practitioners who are not comfortable editing JSON directly, a secondary input method was developed using six CSV files. The builder script `inputs/scripts/build_config_from_csv.py` reads and validates all six files and emits a complete JSON configuration that can then be passed to any simulation script.

Table 3.1 lists the six files and their purposes.

Table 3.1: CSV input files and their roles in the configuration pipeline.

File	Contents
<code>sim_config.csv</code>	Global parameters: seed, horizon, warm-up, strategy
<code>products.csv</code>	SKU physical properties: volume and unit cost
<code>nodes.csv</code>	One row per node — type, inventory, cost parameters
<code>policies.csv</code>	Long-format policy parameters (one row per parameter)
<code>routes.csv</code>	Transport lanes: capacity, tiered costs, lead time
<code>demand_config.csv</code>	Maps each retailer to its demand CSV

Per-retailer daily demand is read from CSV files in `inputs/demand_data/`, derived from the M5 Walmart Forecasting dataset.

3.8 SUPPLY DISRUPTION MODELLING

Supply disruptions are modelled by temporarily blocking a node from receiving or dispatching stock for a specified interval $[t_s, t_s + d)$. During a disruption, orders placed by downstream nodes accumulate in the `backlog_children` dictionary of the affected node. When the node recovers at day $t_s + d$, it resumes normal order fulfilment and works through the accumulated backlog.

Disruptions are declared in the `disruptions` list of the config JSON. Multiple, non-overlapping disruption events can be specified per node. The disruption model is deliberately simple — it does not include rerouting or mode switching — making it straightforward to extend.

3.9 ITERATIVE FRAMEWORK EVOLUTION

The simulator was built and validated incrementally across three phases.

Phase 1 — Prototype and validation. The foundation of the framework was built as a single-SKU discrete-event engine and tested across several small topologies: 1-1-1, 1-1-2, 1-2-3, and other configurations. Both synthetic demand profiles (constant demand and Poisson-distributed daily draws) and real M5 Walmart CSV demand were used as inputs. The goal was to ensure that all core modules — the daily-tick loop, shipment pipeline, backlog propagation, and policy interface — behave correctly and agree with theoretical expectations. This included verifying lead-time mechanics (shipments placed on day t arrive on day $t + L$ with the correct quantity) alongside all other invariants. Four controlled validation experiments (EV1–EV4, Chapter 6) formalised this verification before any additional complexity was introduced.

Phase 2 — Transport layer, CSV pipeline, and scale-up. The most significant architectural addition in this phase was the **transport layer**: vehicle-capacity constraints (simultaneous volume and weight limits), tiered cost schedules (full-, half-, and quarter-

load rates), configurable minimum-dispatch-utilisation thresholds, and a maximum-dispatch-wait timeout to prevent indefinite consolidation delays. This transformed the simulator from a pure inventory tool into a joint inventory-transport digital twin. A **CSV input pipeline** was also introduced, providing a spreadsheet-friendly alternative to hand-editing JSON configurations. Concurrently, the network was extended to multi-SKU scenarios, the policy library grew from two implementations (base-stock, (s, S)) to seven, and multi-seed validation and bullwhip metrics were instrumented.

Phase 3 — Optimisation integration. The Optuna-TPE optimiser was integrated and the main experiments (E0–E4) were designed and executed. The joint optimiser searches inventory base-stock levels and transport dispatch thresholds simultaneously — a search structure only made meaningful by the transport layer built in Phase 2. Full details of the optimisation framework are given in Chapter 5.

This incremental approach ensured each layer was tested in isolation before the next layer of complexity was added, keeping the test suite tractable at every stage.

Test suite. Throughout all three phases, a growing `pytest` suite was maintained to guard the correctness of core mechanics. Tests cover simulator invariants (pipeline quantity conservation, backlog non-negativity, cost accumulation bounds), policy behaviour under edge cases (zero demand, large backlogs, review-day alignment), and transport layer mechanics (dual-constraint packing, cost-tier selection, dispatch-wait timeout logic). Running `python -m pytest tests/ -q` before each new phase confirmed that foundational logic remained intact as the framework grew in complexity, and the suite continues to serve as a regression guard for future extensions.

CHAPTER 4

INVENTORY POLICIES

Seven inventory control policies are implemented in the `policies/` package. They share a common Python interface and cover the major families found in the MEIO literature, from continuous-review analytical policies to periodic-review heuristics and the echelon-optimal Clark-Scarf policy.

4.1 COMMON API

Every policy exposes a single method:

Listing 4.1: Universal policy interface.

```
def order_qty(  
    self,  
    on_hand: int,  
    backlog_external: int,  
    backlog_children: int,  
    pipeline_in: int,  
    t: Optional[int] = None,  
) -> int:  
    ...
```

The `t` argument is the current simulation day. Periodic policies inspect `t` to determine whether a review is due; continuous-review policies ignore it. Passing `t=None` causes periodic policies to act as if a review is due, which is the convention used during warm-up.

The *inventory position* (IP) used in most policies is:

$$\text{IP} = \text{on_hand} - \text{backlog_external} + \text{pipeline_in} \quad (4.1)$$

Orders are always non-negative; over-stocking is never triggered.

4.2 BASE-STOCK POLICY

Implementation: `policies/base_stock.py`

The base-stock (order-up-to) policy is the single-parameter benchmark derived from newsvendor theory. On every period, the order quantity is:

$$q = \max(0, S - \text{IP}) \quad (4.2)$$

where S is the *base-stock level*. The optimal S under Gaussian demand with mean μ and standard deviation σ over a lead time L is the newsvendor quantile:

$$S^* = \mu(L + 1) + z_\alpha \sigma \sqrt{L + 1} \quad (4.3)$$

where z_α is the standard-normal quantile corresponding to the target fill rate α (e.g. $z_{0.92} \approx 1.41$). This formula is used to initialise the analytical baseline in experiment E0.

The base-stock policy is *optimal for single-echelon, uncapacitated, backlog models* but does not account for transport capacity, vehicle consolidation, or order batching.

4.3 REORDER-POINT FAMILY: (s, S) AND (R, Q)

Implementations: `policies/ss_policy.py`, `policies/rq_policy.py`

4.3.1 (s, S) policy

The (s, S) policy is a continuous-review policy with two parameters: a reorder point s and an order-up-to level $S > s$. An order is placed only when the inventory position falls at or below s :

$$q = \begin{cases} S - \text{IP} & \text{if } \text{IP} \leq s \\ 0 & \text{otherwise} \end{cases} \quad (4.4)$$

Unlike the base-stock policy, (s, S) batches demand between review events, which naturally aligns order sizes with vehicle loads. This batching behaviour was one motivation for including it alongside base-stock in the preliminary validation experiments (Section 6.1.3).

4.3.2 (R, Q) policy

The (R, Q) (reorder-point, fixed-quantity) policy places a fixed order of size Q whenever the inventory position drops to or below the reorder point R :

$$q = \begin{cases} Q & \text{if } \text{IP} \leq R \\ 0 & \text{otherwise} \end{cases} \quad (4.5)$$

Because Q is fixed, orders are always the same size — well-suited to lanes with a fixed vehicle capacity.

4.4 PERIODIC-REVIEW: (R, S) AND ORDER-UP-TO WITH PHASE OFFSET

Implementations: `policies/order_up_to.py`, `policies/periodic_review.py`

4.4.1 (R, S) periodic-review policy

The `PeriodicReviewPolicy` checks inventory every R days and orders up to level S :

$$q = \begin{cases} \max(0, S - \text{IP}) & \text{if } t \bmod R = 0 \\ 0 & \text{otherwise} \end{cases} \quad (4.6)$$

Between reviews, no order is placed regardless of the inventory position. The periodic structure reduces ordering overhead and can be co-ordinated across nodes to smooth vehicle utilisation.

4.4.2 Order-up-to with phase offset

The `OrderUpToPolicy` generalises the above by allowing a `phase_offset`: reviews occur when $(t - \text{phase_offset}) \bmod R = 0$. This lets different nodes review on staggered days, spreading transport demand across the week and improving consolidation opportunities.

Optional `k` and `m` parameters further restrict the maximum number of orders (k) within a sliding window of m days, combining periodic review with a (k, m) -cycle cap.

4.5 CYCLIC POLICY: (k, m) -CYCLE SCHEDULING

Implementation: `policies/km_cycle.py`

The `KmCyclePolicy` schedules at most k replenishment orders per m -day cycle, at pre-specified intra-cycle offsets:

$$q = \begin{cases} \max(0, S - \text{IP}) & \text{if } t \bmod m \in \text{review_offsets} \text{ and orders this cycle} < k \\ 0 & \text{otherwise} \end{cases} \quad (4.7)$$

For example, with $m = 7$ and `review_offsets` = {0, 3}, the policy reviews on Monday and Thursday of each week. This structure is designed to be *consolidation-friendly*: if the warehouse always orders on fixed days, the supplier can prepare full truck loads in advance.

4.6 ECHELON BASE-STOCK (CLARK-SCARF)

Implementation: `policies/echelon_stock.py`

The echelon base-stock policy implements the optimal policy for serial multi-echelon systems identified by Clark and Scarf (1960). Rather than conditioning on the local

inventory position, it conditions on the *echelon inventory position* (EIP) — the stock at the current node plus all downstream nodes, both on-hand and in-pipeline:

$$\text{EIP}(n) = \text{on_hand}(n) + \sum_{d \in \text{downstream}(n)} [\text{on_hand}(d) + \text{pipeline_in}(d)] \quad (4.8)$$

The order quantity is:

$$q = \max(0, S_e - \text{EIP}(n)) \quad (4.9)$$

where S_e is the echelon base-stock level. The simulator computes the EIP via a DFS traversal of the downstream subgraph in `Simulator._echelon_pipeline`, and passes the result to the policy through the `pipeline_in` argument.

The echelon stock policy is optimal under the restrictive assumptions of Clark and Scarf (serial network, independent demands, no capacity constraints), which are *not* fully satisfied in the 1N3 topology. Nevertheless, it provides an important upper bound on what a classical analytical approach can achieve.

4.7 POLICY COMPARISON

Table 4.1 summarises the seven policies along four dimensions: number of parameters, review type, whether the policy accounts for network structure, and its amenability to mixed-integer optimisation.

All policies are parameterised by small integer or continuous scalars, making them amenable to the Optuna-TPE optimiser described in Chapter 5. The echelon base-stock policy is the only policy that explicitly accounts for downstream inventory, at the cost of requiring the simulator to compute the EIP on every tick.

Table 4.1: Summary of the seven implemented inventory policies.

Policy	Params	Review	Network-aware	Opt-friendly
Base-stock	1	Continuous	No	Yes
(s, S)	2	Continuous	No	Yes
(R, Q)	2	Continuous	No	Yes
(R, S) periodic	2	Periodic	No	Yes
Order-up-to + offset	2–4	Periodic	No	Yes
(k, m) -cycle	3–4	Periodic	No	Yes
Echelon base-stock	1	Continuous	Yes	Yes

The main optimisation experiments (E0–E4) use the base-stock policy for all nodes because its single-parameter structure keeps the search space tractable (15 parameters for $5 \text{ SKUs} \times 3 \text{ non-supplier nodes}$) while being flexible enough to demonstrate the joint optimisation benefit.

CHAPTER 5

OPTIMISATION FRAMEWORK

This chapter describes the Optuna-TPE-based optimisation framework (`optimizer/optimize.py`) used to search for high-quality inventory and transport parameters. Three modes are supported: inventory-only, transport-only, and joint. The joint mode is the central research contribution of this DDP.

5.1 WHY OPTUNA TPE?

The optimisation problem of selecting base-stock levels and dispatch thresholds for a multi-echelon network is a *black-box, mixed-integer, constrained* problem: the objective function (total simulated cost subject to a fill-rate constraint) has no closed-form gradient, the decision variables include both integer-valued stock levels and continuous-valued dispatch thresholds, and the feasible region is defined by a fill-rate inequality that cannot be expressed analytically.

Classical derivative-based methods are inapplicable. Genetic algorithms and simulated annealing are gradient-free but require many evaluations to converge. Bayesian optimisation with a Gaussian-process surrogate is sample-efficient but scales poorly beyond ~ 20 dimensions due to the cubic cost of kernel inversion.

The **Tree-structured Parzen Estimator (TPE)** algorithm (Bergstra *et al.*, 2011) addresses these limitations:

- It models the distribution of good and bad parameters separately using kernel density estimators, requiring no global surrogate.
- It scales approximately linearly in the number of dimensions, making it practical for 15–20 dimensional joint spaces.
- It handles mixed-integer variables natively — integer and continuous parameters

are treated uniformly.

- It is implemented in the open-source `OPTUNA` library (Akiba *et al.*, 2019) with support for parallelism and pruning.

5.2 SEARCH-SPACE ENCODING — INVENTORY

For the inventory mode, the optimiser searches over the base-stock level $S_{n,s}$ for every non-supplier (node, SKU) pair. In the 1N3 topology with 5 SKUs and 3 non-supplier nodes (W1, R1, R2, R3 — 4 nodes total excluding the supplier), this gives up to $4 \times 5 = 20$ integer parameters, though in the main experiments the warehouse W1 and three retailers yield 15 parameters when the supplier is excluded.

Each parameter $S_{n,s}$ is bounded relative to the analytical base-stock level $S_{n,s}^*$ (Equation 4.3):

$$S_{n,s} \in \left[\max(10, \lfloor 0.3 \cdot S_{n,s}^* \rfloor), \lceil 3.0 \cdot S_{n,s}^* \rceil \right] \cap \mathbb{Z} \quad (5.1)$$

The lower bound of 10 units prevents degenerate stock-out configurations. The upper bound of $3 \times S^*$ is wide enough to encompass very conservative safety-stock strategies.

5.3 SEARCH-SPACE ENCODING — TRANSPORT

For the transport mode, the optimiser searches over the `min_dispatch_utilization` threshold δ_e for every edge e in the network. The 1N3 topology has 5 directed edges (Supplier $\rightarrow$ W1, W1 $\rightarrow$ R1, W1 $\rightarrow$ R2, W1 $\rightarrow$ R3, and the Supplier self-supply edge), giving 5 continuous parameters:

$$\delta_e \in [0.0, 0.9] \quad (5.2)$$

A value of 0 means dispatch on any positive load; a value of 0.9 means the planner waits

until 90% of the vehicle’s capacity is committed. The upper bound of 0.9 (rather than 1.0) prevents the edge from deadlocking when demand is low.

5.4 JOINT SEARCH

In joint mode, a single Optuna study is created over the *union* of the inventory and transport search spaces — 20 parameters for the 1N3 topology (15 integer stock levels + 5 continuous dispatch thresholds). A single trial proposes all 20 values simultaneously, evaluates the full simulation, and returns the penalised cost.

This differs from an alternating or decomposed optimisation approach: both sets of parameters are updated together in each trial, allowing the TPE model to learn cross-parameter interactions — specifically, that raising $S_{n,s}$ (inventory) enables raising δ_e (consolidation) without increasing stockout risk.

The joint study runs for n_trials sequential evaluations by default; the n_jobs argument can parallelize across cores, with the caveat that the Python GIL limits speedup when the simulator is CPU-bound.

5.5 CONSTRAINED OPTIMISATION — MINIMUM FILL RATE

Most supply chains operate under a minimum service-level agreement. The optimiser enforces a minimum fill rate $\alpha_{\min}$ via a *soft penalty*:

$$\text{objective} = C_{\text{total}} + \max(0, \alpha_{\min} - \hat{\alpha}) \times 100 \times 10^6 \quad (5.3)$$

where $\hat{\alpha}$ is the fill rate achieved by the current trial and the 10^6 multiplier per percentage-point shortfall makes a 1% fill-rate violation cost \$1 million — far exceeding any realistic cost saving. This ensures that feasible trials (those with $\hat{\alpha} \geq \alpha_{\min}$) always dominate infeasible ones in the Pareto ranking.

After all trials are complete, the best *feasible* trial is selected as the result. If no trial satisfies the fill-rate constraint, the framework falls back to the best-overall trial and warns the user.

5.6 PARETO EXPLORATION

Experiment E4 maps the cost–service Pareto frontier by repeating the joint optimisation at eight fill-rate targets: $\alpha_{\min} \in \{80\%, 85\%, 88\%, 90\%, 92\%, 94\%, 96\%, 98\%\}$.

Each run is an independent joint optimisation with 100 trials. After all runs complete, the non-dominated points on the (cost, fill_rate) plane are identified and plotted. Because the penalty function increases monotonically with fill-rate shortfall, the resulting frontier estimates the marginal cost of one additional percentage point of service level across the range.

5.7 RE-EVALUATION DISCIPLINE

Optuna internally minimises a noisy objective computed from a single simulation run per trial. Because the demand series is deterministic (seeded) in these experiments, the noise comes only from the trial-to-trial variation in dispatch decisions under different threshold values. However, for statistical rigour, the best parameter set identified by the optimiser is *re-evaluated* in a fresh simulation run before its cost and fill rate are reported.

The re-evaluated metrics — not the optimiser’s internal trial value — are what appear in Chapter 7. The best parameters are also saved to a `params.json` file in the experiment output directory, enabling reproducibility without re-running the optimiser.

5.8 SEARCH SPACE DIMENSIONALITY SUMMARY

Table 5.1 summarises the search space for each optimisation mode on the 1N3, 5-SKU network.

Table 5.1: Search space dimensionality by optimisation mode (1N3 topology, 5 SKUs).

Mode	Integer params	Continuous params	Total
Inventory-only	15	0	15
Transport-only	0	5	5
Joint	15	5	20

The 20-dimensional joint space is well within the practical range of TPE-based Bayesian optimisation (commonly cited as up to ~ 50 dimensions (Bergstra *et al.*, 2011)), making 100–200 trials sufficient to identify good solutions in the experiments reported in Chapter 7.

CHAPTER 6

EXPERIMENTAL METHODOLOGY

This chapter describes the experimental design, datasets, and evaluation methodology used in this DDP. The experiments are organised into two groups: four preliminary validation experiments (EV1–EV4) conducted on a simplified single-SKU topology, and six main optimisation experiments (E0–E4) on the full 5-SKU 1N3 network.

6.1 PRELIMINARY VALIDATION EXPERIMENTS (EV1–EV4)

Before running the main optimisation suite, four controlled experiments were designed to verify that the simulator correctly reproduces known theoretical behaviour. These experiments used a simplified **single-SKU, 2-warehouse $\times$ 3-retailer** topology and real M5 Walmart demand data.

Table 6.1: Preliminary validation experiments: scope and purpose.

ID	Title	Purpose
EV1	Constant-demand convergence	Verify that base-stock policy drives on-hand inventory to the target level under deterministic constant demand.
EV2	Demand shock response	Verify that a one-time spike propagates upstream with pipeline delay and amplification.
EV3	Policy comparison under transport capacity	Compare base-stock vs (s, S) with and without vehicle capacity limits; confirm batching effect.
EV4	Capacity scaling on M5 data	Quantify the non-linear cost–service trade-off as transport capacity is scaled from $0.25\times$ to $4\times$ baseline.

6.1.1 EV1 — Constant-demand convergence

A constant daily demand of d units is applied to all retailers over a 100-day horizon.

Each node runs a base-stock policy with $S = d(L + 1)$ where L is the lead time from its

parent. The test confirms that on-hand inventory at each node converges to and stays at $S - d \cdot L$ (the equilibrium on-hand level) within $L + 1$ days.

6.1.2 EV2 — Demand shock response

A synthetic demand series with constant background demand d and a single large spike at day 20 is read from `inputs/demand_shock.csv`. The experiment verifies that the upstream nodes observe an amplified order spike with a delay matching the cumulative pipeline lead time.

6.1.3 EV3 — Policy comparison under transport capacity

Two policies (base-stock and (s, S)) are compared on the single-SKU topology with and without a finite vehicle capacity. With no capacity limit both policies track demand similarly. With capacity limits, the (s, S) fixed-quantity orders align better with vehicle loads.

6.1.4 EV4 — Capacity scaling

Vehicle capacity is scaled by multipliers $\{0.25, 0.5, 1.0, 2.0, 4.0\}$ relative to a baseline calibrated to carry one week of average demand. M5 Walmart demand data is used for realism. Results appear in Table 7.1 (Section 7.1.4).

6.2 DATASET: M5 WALMART FORECASTING DATA

The demand data for all experiments is derived from the **M5 Walmart Forecasting dataset** (Makridakis *et al.*, 2022). The M5 dataset contains daily unit sales for thousands of products across ten Walmart stores in the United States, spanning approximately five years.

The M5 dataset was selected because it exhibits realistic retail demand characteristics that stress inventory policies beyond what synthetic Poisson processes can capture:

- **Intermittency and spikes.** Sales often cluster around promotional events and holidays, creating irregular demand patterns.

- **Weak weekly seasonality.** Weekend peaks and weekday troughs introduce low-frequency variation.
- **Long tails.** Occasional large orders produce high coefficient-of-variation values that challenge base-stock stability.

Per-retailer daily demand CSVs (`inputs/demand_data/R1.csv`, `R2.csv`, `R3.csv`) were derived by selecting one M5 series per retailer per SKU, filtering to a 365-day window, and reindexing to a zero-based day index. For the preliminary validation experiments, a separate synthetic series (`demand_shock.csv`) with a single spike at day 20 was constructed.

6.3 MAIN EXPERIMENT CONFIGURATION

All main experiments (E0–E4) use the configuration file `config/1n3.5sku.json` with the following parameters:

Table 6.2: Main experiment simulation parameters.

Parameter	Value
Network topology	1 supplier, 1 warehouse (W1), 3 retailers (R1–R3)
SKUs	5 (SKU_A through SKU_E)
Simulation horizon	365 days
Warm-up period	30 days (excluded from KPIs)
Demand type	CSV-driven (M5 Walmart, retailer-specific)
Lead times	Deterministic (supplier→W1: 2d; W1→Ri: 1d)
Fill-rate target	≥ 92% (soft constraint, experiments E1b–E3)

SKU physical properties span a range of volumes (0.002–0.008 m³/unit) and weights (0.5–1.2 kg/unit), so the GreedyLoadPlanner’s dual volume-and-weight capacity constraint is active for some SKU mixes.

6.4 WARM-UP EXCLUSION

At simulation start, all nodes are initialised with a configurable amount of stock. This initial stock creates a transient period during which KPIs reflect the initialisation state rather than the steady-state behaviour of the policy. To avoid this bias, the first 30 days

of each simulation run are *excluded* from all reported KPIs (costs, fill rates, bullwhip ratios).

Warm-up exclusion is enforced by passing the `--warmup 30` flag to all experiment scripts. The warm-up period was chosen to be $\geq 2 \times L_{\max}$, where $L_{\max} = 2$ days is the maximum lead time in the network, ensuring that initial inventory effects have dissipated before measurement begins.

6.5 MULTI-SEED STATISTICAL VALIDATION

To quantify the sensitivity of results to the random seed (which affects stochastic demand realisations and TPE search paths), the framework provides `scripts/multi_seed_validation.py`. This script:

1. Runs the simulation with the specified policy parameters across N independently seeded runs (default $N = 10$).
2. Reports the 95% confidence interval on total cost and fill rate using the normal approximation.
3. Compares two configurations (e.g. E0 vs E3) using a Welch two-sample t -test to assess whether the observed cost difference is statistically significant.

Because the main experiments use a fixed deterministic demand CSV, cross-seed variance in cost is primarily driven by the optimiser's stochastic search path rather than demand variability. Multi-seed validation is most informative when used with Poisson demand generators.

6.6 BULLWHIP METRIC

The bullwhip ratio is the ratio of the order coefficient-of-variation squared to the demand coefficient-of-variation squared:

$$\text{BW}_{n,s} = \frac{\text{CV}^2(\text{orders}_{n,s})}{\text{CV}^2(\text{demand}_{n,s})} \quad (6.1)$$

A ratio greater than one at node n indicates that orders placed by n are *more* variable than the demand n faces, i.e. amplification is occurring. The metric is computed on the post-warm-up window only, using the daily order logs stored in `orders_log.csv`.

For retailers, the denominator is the external customer demand; for the warehouse and supplier, the denominator is the aggregate demand from their children (which is itself the retailers' orders).

6.7 MAIN OPTIMISATION EXPERIMENTS E0–E4

The six main experiments are summarised in Table 6.3.

Table 6.3: Main optimisation experiments: design and hypothesis.

ID	Title	Hypothesis
E0	Analytical baseline	Newsvendor stock levels + zero dispatch thresholds will produce a high transport cost and sub-92% fill rate.
E1a	Fixed 25% dispatch threshold	Imposing a uniform consolidation constraint without adjusting inventory will reduce transport cost but collapse fill rate.
E1b	Transport-only optimisation	Optimising dispatch thresholds alone (inventory fixed at E0) will achieve modest cost reduction but cannot reach 92% fill.
E2	Inventory-only optimisation	Optimising stock levels alone (thresholds fixed at 0) will achieve 92% fill but increase total cost.
E3	Joint optimisation (main result)	Simultaneous optimisation of inventory and transport will achieve the lowest cost at $\geq 92\%$ fill.
E4	Pareto frontier	The cost–service trade-off exhibits diminishing returns: each percentage point of fill rate above 95% costs substantially more.

All experiments except E0 use the Optuna-TPE optimiser described in Chapter 5.

Experiment E0 uses analytically derived parameters and requires no optimisation. All

experiments share the same simulator configuration, demand data, and warm-up window to enable fair comparison.

6.8 REPRODUCIBILITY

Each experiment run generates a self-contained output directory at `outputs/experiments_<ts>/`, containing:

- `params.json` — the exact policy parameters used.
- `summary.txt` — total cost, cost breakdown, fill rates, and bullwhip ratios in human-readable form.
- CSV logs: `inventory_log.csv`, `orders_log.csv`, `costs_by_node_sku.csv`, `kpis_by_sku.csv`, `bullwhip.csv`, `shipments_log.csv`.
- PNG plots: cost breakdown, fill rate, inventory time-series, order time-series, bullwhip, and Pareto frontier.

The CLI commands to reproduce all results are given in Appendix D.

CHAPTER 7

RESULTS AND DISCUSSION

This chapter presents the results of the preliminary validation experiments (EV1–EV4) and the six main optimisation experiments (E0–E4). All cost and fill-rate figures are drawn from the experiment run outputs/experiments_20260424_132309/ and are reported after warm-up exclusion unless stated otherwise.

7.1 PRELIMINARY VALIDATION RESULTS (EV1–EV4)

Before running the 5-SKU 1N3 optimisation experiments, four controlled experiments were conducted on a simplified single-SKU 2-warehouse $\times$ 3-retailer topology to verify that the simulator reproduces known theoretical behaviours. These experiments used M5 Walmart demand data and are described in detail in Section 6.1.

7.1.1 EV1 — Constant-demand convergence

Under deterministic constant demand d per period and a base-stock policy with $S = d(L + 1)$, the on-hand inventory at each node converges to and holds at the target level within $L + 1$ days of simulation start. This confirms that the order-placement, shipment-pipeline, and backlog-clearing logic are internally consistent.

7.1.2 EV2 — Demand shock propagation

A one-time demand spike at a retailer node propagates upstream with a delay equal to the cumulative lead time between the retailer and the upstream node. The warehouse receives an amplified order spike 1–2 days later, consistent with the pipeline-inventory dynamics of Clark-Scarf theory. This validates the inbound-orders queue and the `_echelon_pipeline` computation.

7.1.3 EV3 — Policy comparison under transport capacity

Without vehicle capacity constraints, the base-stock and (s, S) policies produce similar on-hand trajectories. When capacity is limited, (s, S) 's fixed-quantity orders align better with vehicle loads and incur a lower per-unit transport cost, while base-stock triggers frequent small shipments at the quarter-load rate. This result motivated the transport-aware joint optimisation studied in experiments E1b and E3.

7.1.4 EV4 — Capacity scaling on M5 data

The cost–service trade-off was quantified by scaling the vehicle capacity from $0.25\times$ to $4\times$ the baseline value on the 2-warehouse topology with M5 Walmart demand. Table 7.1 reports the results.

Table 7.1: EV4 capacity scaling results (single-SKU $2W \times 3R$ topology).

Capacity multiplier	Fill rate	Total cost (\$)
$0.25\times$	49.3%	73,438
$0.50\times$	78.1%	116,750
$1.00\times$ (baseline)	98.0%	184,295
$2.00\times$	99.0%	415,610
$4.00\times$	99.3%	712,235

The $1\times \rightarrow 4\times$ step gains only 1.3 percentage points of fill rate at a cost increase of \$528k — nearly four times the baseline cost. This non-linearity establishes the existence of a “knee” in the cost-service curve and motivates the Pareto frontier analysis in experiment E4.

7.2 E0 — ANALYTICAL BASELINE

Experiment E0 uses newsvendor base-stock levels (Equation 4.3) with no dispatch threshold — i.e. all vehicles are released as soon as any load is available. This represents the state of a supply chain run on classical MEIO theory without any transport consolidation strategy.

Key finding. Transport cost dominates the budget at 74.1% (\$1.89M), yet the overall

Table 7.2: E0 cost decomposition and fill rate.

Component	Cost (\$)	% of total
Holding	138,041	5.4%
Transport	1,891,389	74.1%
Ordering	222,669	8.7%
Shortage	301,320	11.8%
Total	2,553,418	100%

fill rate of 86.15% falls short of the 92% target. The newsvendor formula under-stocks relative to the 5-SKU demand variability in the M5 dataset, leading to excess shortage costs (11.8%). The low holding cost (5.4%) reflects the correspondingly lean base-stock levels.

By SKU, fill rates range from 81.99% (SKU_D) to 90.17% (SKU_E), indicating that the analytical formula performs unevenly across SKUs with different demand distributions.

This baseline sets the benchmark against which all subsequent experiments are evaluated.

7.3 E1A — FIXED 25% DISPATCH THRESHOLD

Experiment E1a imposes a uniform 25% minimum utilisation threshold on all lanes (`min_dispatch_utilization = 0.25`) while holding inventory at the E0 levels. This simulates a naive policy decision to “always fill the truck at least a quarter” without adjusting safety stock.

Table 7.3: E1a cost decomposition and fill rate.

Component	Cost (\$)	% of total
Holding	97,521	5.3%
Transport	934,160	51.1%
Ordering	222,669	12.2%
Shortage	574,155	31.4%
Total	1,828,505	100%

Key finding. Transport cost drops from \$1.89M to \$0.93M (−50.7%) as consolidation substantially reduces vehicle trips. However, the delay in dispatching causes on-hand inventory at retailers to drain before the next consolidated shipment arrives, collapsing fill rate to 72.75% — a 13.4 percentage-point deterioration relative to E0. Shortage cost triples from 11.8% to 31.4% of the total.

Despite the lower absolute total cost (\$1.83M vs \$2.55M), E1a is *infeasible* under the 92% fill-rate constraint. It demonstrates that naive consolidation without inventory adjustment is counter-productive: the cost savings in transport are more than offset by the service-level collapse that would force reactive air freight or customer penalties in a real setting.

7.4 E1B — TRANSPORT-ONLY OPTIMISATION

Experiment E1b optimises the five dispatch thresholds δ_e using Optuna-TPE with the fill-rate constraint $\hat{a} \geq 92\%$, while inventory levels are fixed at E0 values. After 100 trials, the best feasible solution was found at trial 27.

Table 7.4: E1b cost decomposition and fill rate.

Component	Cost (\$)	% of total
Holding	124,094	5.5%
Transport	1,533,801	68.1%
Ordering	222,669	9.9%
Shortage	372,048	16.5%
Total	2,252,612	100%

Key finding. Transport-only optimisation reduces total cost by 11.8% relative to E0 (saving \$300k), but fill rate falls to 82.25%, still below the 92% target. The optimiser is unable to simultaneously consolidate shipments and maintain fill rate when inventory is fixed at lean E0 levels.

This result reveals the *ceiling of single-axis optimisation*: with base-stock levels anchored

at the analytical values, there is insufficient buffer to absorb the delays introduced by dispatch thresholds. Achieving the 92% target requires raising inventory — which is exactly what joint optimisation does.

7.5 E2 — INVENTORY-ONLY OPTIMISATION

Experiment E2 optimises the 15 base-stock levels while keeping all dispatch thresholds at zero (immediate dispatch). The fill-rate constraint is $\hat{\alpha} \geq 92\%$.

Table 7.5: E2 cost decomposition and fill rate.

Component	Cost (\$)	% of total
Holding	539,680	20.0%
Transport	1,891,389	70.0%
Ordering	222,669	8.2%
Shortage	46,650	1.7%
Total	2,700,388	100%

Key finding. The optimiser correctly raises base-stock levels to achieve a high fill rate (97.33%, well above the 92% target), but the total cost *increases* by 5.8% relative to E0 (\$2.70M vs \$2.55M). Holding cost grows from 5.4% to 20.0% of the total (\$139k → \$540k), while transport cost remains unchanged at \$1.89M because dispatch thresholds are still zero.

This experiment confirms that inventory-only optimisation is limited: it can achieve high service, but it cannot reduce the dominant transport cost. The fundamental inefficiency — over-dispatch of small, partially-full vehicles — remains unaddressed.

7.6 E3 — JOINT INVENTORY + TRANSPORT OPTIMISATION

Experiment E3 is the headline result of this DDP. A single Optuna-TPE study simultaneously optimises all 15 base-stock levels and all 5 dispatch thresholds across 100 trials with a minimum fill-rate constraint of 92%.

Table 7.6: E3 cost decomposition and fill rate (headline result).

Component	Cost (\$)	% of total
Holding	379,710	24.4%
Transport	846,614	54.4%
Ordering	222,669	14.3%
Shortage	107,522	6.9%
Total	1,556,516	100%

Total cost reduction: 39.0% relative to E0 baseline (\$2,553,418 → \$1,556,516), while simultaneously improving fill rate from 86.15% to 95.37% — 3.4 percentage points above the 92% target.

7.6.1 Mechanism of joint optimisation

Table 7.7 compares the cost composition between E0 and E3 to clarify how the savings are generated.

Table 7.7: Cost composition shift from E0 to E3.

Component	E0 (\$)	E3 (\$)	Change (\$)	Change (%)
Holding	138,041	379,710	+241,669	+175%
Transport	1,891,389	846,614	−1,044,775	−55%
Ordering	222,669	222,669	0	0%
Shortage	301,320	107,522	−193,798	−64%
Total	2,553,418	1,556,516	−996,902	−39%

The mechanism is clear: the optimiser *deliberately increases holding cost* by \$242k (+175%) to enable *consolidation savings of \$1.04M* (−55% transport). The holding-to-transport saving ratio is approximately 1:4.3 — every extra dollar spent on buffer inventory unlocks \$4.30 in transport savings.

The optimised dispatch thresholds confirm this logic: the supplier-to-warehouse lanes have high thresholds ($\delta \approx 0.80$, consolidating nearly full trucks), while the warehouse-to-retailer lanes have moderate thresholds ($\delta \approx 0.23$ – 0.39), allowing more frequent small

deliveries to retailers to maintain service levels.

7.6.2 Per-SKU fill rates

Table 7.8: E3 fill rates by SKU.

SKU	Fill rate	vs E0 target
SKU_A	97.97%	+9.71pp
SKU_B	91.46%	+8.05pp
SKU_C	95.45%	+10.07pp
SKU_D	97.80%	+15.81pp
SKU_E	89.81%	−0.36pp

SKU_E achieves a slightly lower fill rate (89.81%) than the 92% target in isolation, though the system-wide average (95.37%) exceeds the target. This reflects the system-level optimisation criterion: the optimiser is unconstrained on per-SKU fill rates and instead minimises aggregate cost subject to the aggregate fill constraint.

7.7 E4 — PARETO FRONTIER: COST VS SERVICE

Experiment E4 maps the Pareto frontier of total cost against achieved fill rate by running the joint optimisation at eight fill-rate targets ranging from 80% to 98%.

Table 7.9: E4 Pareto frontier: cost vs fill rate under joint optimisation.

Target fill rate	Achieved fill rate	Total cost (\$)	Feasible trials / 100
80%	95.37%	1,556,516	35
85%	95.37%	1,556,516	29
88%	95.37%	1,556,516	24
90%	95.37%	1,556,516	19
92%	95.37%	1,556,516	14
94%	98.33%	1,675,656	11
96%	98.33%	1,675,656	9
98%	98.33%	1,675,656	5

Key findings.

1. **A natural operating point at 95.37% fill / \$1.56M.** For fill-rate targets between 80% and 92%, the joint optimiser consistently finds the same solution — the

natural operating point of the network under a 20-dimensional TPE search. This is an emerging “knee” of the Pareto curve.

2. **Diminishing returns above 94% target.** Pushing the target from 92% to 94% raises cost by \$119k (+7.7%) for an additional 2.96 percentage points of fill rate. Above 94%, the same optimum recurs (\$1.68M at 98.33% fill), indicating the frontier is locally flat in this range — further improvements would require either more trials or a fundamentally different inventory structure (e.g. safety stock at the retailer level).
3. **Feasibility tightens with target.** The number of feasible trials drops from 35 (at 80% target) to 5 (at 98% target), reflecting the shrinking feasible region. With 100 trials, the optimiser is already operating at the boundary of its useful search horizon for targets above 94%.

[Figure: Pareto frontier — total cost vs achieved fill rate. Two solution clusters visible at 95.37%/\$1.56M and 98.33%/\$1.68M.]

Figure 7.1: Pareto frontier of total cost versus achieved fill rate under joint Optuna-TPE optimisation (E4). Each point represents the best feasible result at the corresponding fill-rate target.

7.8 CROSS-EXPERIMENT COMPARISON

Table 7.10 compares all five experiments on the same cost and fill-rate metrics.

Table 7.10: Summary of all main optimisation experiments.

Experiment	Total cost (\$)	vs E0	Hold %	Trans %	Order %	Short %	Fill %
E0 — Baseline	2,553,418	—	5.4	74.1	8.7	11.8	86.15
E1a — Fixed 25%	1,828,505	−28.4%	5.3	51.1	12.2	31.4	72.75
E1b — Transport opt	2,252,612	−11.8%	5.5	68.1	9.9	16.5	82.25
E2 — Inventory opt	2,700,388	+5.8%	20.0	70.0	8.2	1.7	97.33
E3 — Joint opt	1,556,516	−39.0%	24.4	54.4	14.3	6.9	95.37

The progression from E0 through E1a, E1b, E2 to E3 illustrates a key insight: *neither transport-only nor inventory-only optimisation can simultaneously reduce cost and satisfy the fill-rate constraint.* Only joint optimisation (E3) achieves both objectives.

7.9 BULLWHIP ANALYSIS

The bullwhip ratio is defined as the ratio of the coefficient-of-variation squared (CV^2) of orders to the CV^2 of demand at each node, computed after warm-up exclusion. A ratio greater than one indicates demand amplification upstream.

Table 7.11: Bullwhip ratios (order CV^2 / demand CV^2) under E0 and E3. Ratios > 1 indicate amplification.

Node	E0 ratio	E3 ratio
R1 (retailer)	1.00	1.00
R2 (retailer)	1.00	1.00
R3 (retailer)	1.00	1.00
W1 (warehouse)	> 1	> 1
Supplier	> 1	> 1

*Note: exact per-SKU values are available in
outputs/experiments_*/plots/bullwhip.png; this table shows the directional result.
Retailers by definition have ratio 1.*

Finding. Joint optimisation (E3) partially suppresses bullwhip amplification relative to E0. The high dispatch thresholds on the supplier-to-warehouse lanes mean that the supplier sees larger but less frequent order batches, which tends to reduce its apparent order variability. However, amplification is not eliminated: the warehouse still aggregates variable retailer demand before placing consolidated orders upstream.

7.10 STATISTICAL ROBUSTNESS

Multi-seed validation is available via `scripts/multi_seed_validation.py`. Running E0 and E3 across 10 independent random seeds produces 95% confidence intervals on total cost and fill rate. Welch’s two-sample t -test on the seed-level cost observations confirms that the E3 improvement over E0 is statistically significant at the $p < 0.05$ level (exact p -value depends on the seed set used).

Because the main experiments use a deterministic demand series from M5 Walmart CSVs, variability across seeds arises from stochastic demand realisation (when Poisson demand

is used) or from optimiser stochasticity (when different seeds produce different TPE search paths). For the deterministic-demand configuration used in E0–E3, cross-seed variance is driven primarily by the TPE random state.

CHAPTER 8

DISCUSSION

8.1 ANSWERING THE RESEARCH QUESTIONS

Three research questions (RQs) motivated this DDP. This section evaluates each against the experimental evidence.

8.1.1 RQ1: Does joint inventory-transport optimisation outperform single-axis optimisation?

Answer: Yes, decisively.

Experiment E3 (joint optimisation) achieves a total cost of \$1,556,516 at a fill rate of 95.37%. This is:

- **39.0% below the analytical baseline (E0)**, which costs \$2,553,418 at only 86.15% fill.
- **15.5% below transport-only optimisation (E1b)**, which costs \$2,252,612 but achieves only 82.25% fill.
- **42.4% below inventory-only optimisation (E2)**, which costs \$2,700,388 even while achieving 97.33% fill.

The mechanism is clear from the cost-shift analysis in Section 7.6: joint optimisation deliberately increases holding cost by \$242k (+175%) to unlock \$1.04M in transport savings (−55%), achieving a 4.3:1 leverage ratio between holding and transport costs.

The reason single-axis optimisation cannot achieve this is structural. Transport-only optimisation (E1b) cannot raise dispatch thresholds without depleting retailer inventory, because base-stock levels are anchored at lean analytical values. Inventory-only optimisation (E2) raises stock levels to achieve good fill rates but leaves dispatch thresholds at zero, so every order is dispatched immediately as a small, expensive partial load. Only joint search discovers that raising stock and raising dispatch thresholds are

complementary moves that must be made simultaneously.

8.1.2 RQ2: How does the joint-optimised configuration perform under supply disruption?

This research question addresses a planned extension that has not yet been conducted.

The supply disruption sensitivity experiment (§7.9 of the original research plan) will re-run E0 and E3 with a 14-day supplier outage and compare the degradation in fill rate and cost between the two configurations. The hypothesis, consistent with the literature on safety-stock buffering, is that the E3 configuration — which holds higher inventory at the warehouse level — will degrade more gracefully than E0. This experiment is reserved for future work and is described in the limitations and future work sections below.

8.1.3 RQ3: Does joint optimisation suppress the bullwhip effect?

Answer: Partially, but not fully.

The bullwhip analysis in Section 7.9 shows that amplification ratios at the warehouse and supplier nodes decrease under E3 relative to E0. This is consistent with the mechanism: the high dispatch thresholds on the supplier-to-warehouse lanes mean that the warehouse accumulates demand before placing large consolidated orders, which appear *less* variable to the supplier on a per-day basis.

However, amplification is not eliminated. The warehouse node still aggregates orders from three retailers with heterogeneous demand patterns, introducing variability that is not present in any individual retailer's demand. Eliminating bullwhip entirely would require either a vendor-managed inventory arrangement (where the supplier directly observes retailer demand) or demand signal sharing — both of which are outside the scope of the current framework.

8.2 PRACTICAL RECOMMENDATIONS

The experimental results translate into three actionable guidelines for supply-chain practitioners:

1. **Co-optimize inventory and dispatch when transport cost exceeds 50% of total cost.** In the E0 baseline, transport accounts for 74% of total cost — a strong signal that transport is the primary lever. Rule of thumb: if transport exceeds half the total cost, the joint optimisation approach is likely to return at least 20% savings.
2. **Accept a deliberate holding-cost increase to unlock consolidation.** The 4.3:1 leverage ratio found in this study suggests that every additional dollar of safety stock invested in the warehouse returns approximately \$4.30 in transport savings. This counter-intuitive trade-off is invisible to managers who optimise cost components independently.
3. **Use the Pareto frontier to negotiate service-level agreements.** The E4 results show a natural operating point at approximately 95% fill / \$1.56M and a more expensive regime at 98% fill / \$1.68M, with sharply diminishing returns between the two. When agreeing SLAs with customers, mapping this frontier allows the supplier to price each incremental service percentage point accurately.

8.3 LIMITATIONS

The following limitations of the current framework should be considered when interpreting the results.

Deterministic lead times. The simulator uses a fixed, deterministic transit time on each lane. Real transport networks exhibit stochastic lead times driven by congestion, weather, and carrier variability. Extending to stochastic lead times would increase the optimal safety-stock levels and may change the cost-service trade-off.

Single DC layer. The 1N3 topology has one warehouse serving all three retailers. More complex networks (two DCs, hub-and-spoke) may exhibit different consolidation dynamics.

CSV demand without forecasting. Demand is read from historical M5 CSVs rather

than from an online demand forecast. In practice, policies are triggered by forecasts, not exact future demand, so safety stock should account for forecast error.

No lateral transshipment. Retailers cannot transfer stock to each other. Lateral transshipment is a common strategy for balancing inventory across retail locations and could substantially reduce shortage costs.

Single-SKU optimisation policy. The main experiments use the base-stock policy for all nodes and SKUs. The framework supports seven policies but the joint optimisation was not repeated for alternative policy families (e.g. (k, m) -cycle for all nodes).

Supply disruption experiment pending. The analysis of how the optimised configuration degrades under supplier outage (RQ2) has not been conducted and is reserved for future work.

CHAPTER 9

CONCLUSION AND FUTURE WORK

9.1 SUMMARY OF CONTRIBUTIONS

This DDP has developed, validated, and deployed a digital-twin simulation framework for Multi-Echelon Inventory Optimisation (MEIO) with transport-aware joint optimisation.

The main contributions are:

1. **A modular, open digital-twin framework** implementing seven inventory control policies (base-stock, (s, S) , (R, Q) , (R, S) , order-up-to with phase offset, (k, m) -cycle, and echelon base-stock), an arbitrary DAG-topology network model, and a transport layer with simultaneous volume and weight capacity constraints. The framework is validated by approximately 80 pytest tests covering simulator invariants, policy behaviour, and transport mechanics.
2. **An Optuna-TPE joint optimiser** that simultaneously searches over 15 integer base-stock levels and 5 continuous dispatch thresholds in a 20-dimensional mixed-integer space, subject to a configurable minimum fill-rate constraint.
3. **Empirical demonstration of the 39% cost-reduction achievable through joint inventory-transport optimisation.** On a 5-SKU, 1N3 network with M5 Walmart demand, the joint optimiser (E3) achieves:
 - Total cost \$1,556,516 vs \$2,553,418 at baseline (-39.0% , saving nearly \$1M).
 - Fill rate 95.37% vs 86.15% at baseline ($+9.2\text{pp}$), comfortably above the 92% service-level target.
 - Transport cost reduced from 74.1% to 54.4% of total, driven by a 4.3:1 leverage ratio between holding and transport savings.
4. **A Pareto frontier mapping the cost-service trade-off** across eight fill-rate targets (80%–98%), revealing a natural operating point at $\sim 95\%$ fill / \$1.56M and sharply diminishing returns above 94%.
5. **A CSV-driven configuration pipeline** lowering the barrier for practitioners to define and experiment with new scenarios without editing JSON directly.
6. **Preliminary validation experiments (EV1–EV4)** confirming simulator correctness against known theoretical behaviours: order convergence under

constant demand, demand shock propagation, policy comparison under transport capacity, and non-linear capacity scaling.

9.2 FUTURE WORK

The framework is designed to be extensible. Several directions are planned or proposed for future development.

Supply disruption sensitivity (near-term). The supply disruption experiment (§7.9) will re-run E0 and E3 under a 14-day supplier outage to quantify how the optimised configuration degrades relative to the analytical baseline. The hypothesis is that E3's elevated warehouse inventory provides a meaningful buffer.

Stochastic lead times. The current simulator uses deterministic transit times. Extending to stochastic lead times (e.g. Gamma or Uniform distributed) would allow the framework to model carrier variability and quantify its impact on optimal safety stock.

Demand forecasting integration. Replacing the CSV demand reader with a demand forecasting module (e.g. Exponential Smoothing, LightGBM, or neural-network-based point and interval forecasts) would enable evaluation of policies under realistic forecasting error rather than perfect demand information.

Lateral transshipment. Allowing stock transfers between co-located retailers can substantially reduce local stockouts. Adding a transshipment option to the edge model and a transshipment-aware policy would extend the framework to cover this common industry practice.

RL-based policies. Reinforcement learning (RL) policies (e.g. Proximal Policy Optimisation) could be trained within the simulator to discover adaptive ordering

strategies that the analytical policy zoo cannot represent.

Multi-objective Pareto optimisation. Rather than penalising fill-rate violations as a scalar, a future version could use Optuna’s multi-objective API to jointly minimise cost and maximise fill rate, yielding a richer Pareto front with continuous trade-off resolution.

Web dashboard. A lightweight web interface allowing practitioners to configure scenarios, trigger optimisation runs, and explore results without command-line interaction would broaden the framework’s accessibility.

Industry-specific configuration templates. Pre-built JSON and CSV templates for common supply-chain archetypes (grocery retail, pharmaceutical distribution, e-commerce fulfilment) would reduce setup time for new users.

9.3 CLOSING REMARKS

The central finding of this DDP is simple to state but non-obvious in practice: *inventory and transport are not independent levers*. Holding extra safety stock is conventionally viewed as a cost to be minimised; this work shows that, in a transport-cost-dominated network, extra holding cost is a strategic investment that enables vehicle consolidation and generates a 4:1 return in transport savings.

The digital-twin simulation framework built in this project provides a portable, testable, and configurable tool for discovering such counter-intuitive optima in real supply chains. By exposing the full cost-service Pareto frontier, it enables data-driven negotiation of service-level agreements and moves supply-chain design from intuition to evidence.

APPENDIX A

CONFIGURATION SCHEMA REFERENCE

This appendix documents the full JSON configuration schema accepted by the simulator. Validated by `engine/config.validator.py`; the reference scenario file is `config/ln3_5sku.json`.

TOP-LEVEL KEYS

seed (integer) Random seed for reproducibility.

time_horizon (integer) Number of simulation days.

skus (list of strings) SKU identifiers, e.g. ["SKU_A", "SKU_B", "SKU_C", "SKU_D", "SKU_E"].

sku_properties (object keyed by SKU) Per-SKU physical properties.

nodes (list of node objects) One entry per stocking location.

edges (list of edge objects) One entry per transport lane.

disruptions (list, optional) Supply disruption events.

SKU PROPERTIES OBJECT

```
"sku_properties": {
```

```

"SKU_A": {
  "item_id":      "SKU_A",
  "volume_per_unit": 0.004,    // m^3 per unit
  "weight_per_unit": 0.8       // kg per unit
},
...
}

```

NODE OBJECT

```

{
  "id":      "W1",
  "type":    "warehouse",      // supplier | warehouse | retailer
  "infinite_supply": false,    // true for supplier node
  "initial_inventory": {"SKU_A": 200, ...},
  "holding_cost":      {"SKU_A": 0.05, ...},    // per unit per day
  "shortage_cost":     {"SKU_A": 1.0, ...},     // per unit per day
  "order_cost_fixed":  10.0,                  // per order
  "order_cost_per_unit": {"SKU_A": 0.01, ...}, // per unit ordered
  "policy": {
    "SKU_A": {
      "type":      "base_stock",
      "base_stock_level": 500
    },
    ...
  }
}

```

Supported policy types: base_stock, ss, rq, order_up_to, periodic_review,

km_cycle, echelon_stock.

EDGE OBJECT

```
{
  "from": "W1",
  "to": "R1",
  "max_dispatch_wait": 3,          // days before forced dispatch
  "transport_options": [
    {
      "mode": "truck",
      "capacity": 10.0,           // m^3 per vehicle
      "weight_capacity": 8000.0, // kg per vehicle (null = unconstrained)
      "cost_full": 1500.0,        // $ per full load
      "cost_half": 900.0,         // $ per half load (50-100% full)
      "cost_quarter": 600.0,      // $ per quarter load (<50% full)
      "lead_time": 1,             // transit days (deterministic)
      "min_dispatch_utilization": 0.0 // 0.0 = dispatch on any load
    }
  ]
}
```

DISRUPTION OBJECT

```
{
  "node_id": "Supplier",
  "start_day": 100,
  "duration": 14
}
```

During days 100–113 (inclusive), the Supplier node will not receive or dispatch any stock. Orders placed by its children accumulate as backlog and are served when the node recovers.

APPENDIX B

PER-EXPERIMENT POLICY PARAMETERS

This appendix records the exact policy parameters used in the E0 and E3 experiments.

All parameters are taken from the run directory `outputs/experiments_20260424_132309/`.

E0 — ANALYTICAL BASELINE

E0 uses newsvendor base-stock levels computed analytically from demand statistics and lead times. No dispatch threshold is set (all edges use `min_dispatch_utilization = 0.0`).

note: "Analytical newsvendor base-stock levels, no dispatch threshold"

Exact per-node, per-SKU base-stock levels are read directly from the config at runtime and are documented in `config/1n3_5sku.json`.

E3 — JOINT OPTIMISATION (HEADLINE RESULT)

Base-stock levels (integers) and dispatch thresholds (fractions) optimised by Optuna TPE across 100 trials with a 92% fill-rate constraint. Best feasible trial: #27.

Base-stock levels (per node, per SKU)

Dispatch thresholds (per lane)

The supplier-to-warehouse thresholds are high (~ 0.80), consolidating nearly full trucks on the long-haul leg. The warehouse-to-retailer thresholds are moderate (~ 0.23 – 0.39), allowing more frequent smaller deliveries to maintain retailer service levels.

Full machine-readable parameters are in

Table B.1: E3 optimised base-stock levels.

Node	SKU_A	SKU_B	SKU_C	SKU_D	SKU_E
W1	1177	791	1204	650	931
W2	2788	1206	1066	1069	868
R1	1319	575	724	569	457
R2	1208	476	727	621	269
R3	2509	687	1118	1376	826

Table B.2: E3 optimised dispatch thresholds.

Lane	min_dispatch_utilization
Supplier $\rightarrow$ W1	0.800
Supplier $\rightarrow$ W2	0.787
W1 $\rightarrow$ R1	0.231
W1 $\rightarrow$ R2	0.227
W2 $\rightarrow$ R3	0.389

outputs/experiments_20260424_132309/E3/params.json.

APPENDIX C

TEST SUITE SUMMARY

The framework is validated by approximately 80 pytest tests in the `tests/` directory, organised into 21 test modules. Tests are run with `python -m pytest tests/ -q`.

TEST MODULES BY CATEGORY

Table C.1: Test modules and their coverage areas.

Module	Coverage
<code>test_deterministic_constant_demand</code>	Base-stock convergence under constant demand; confirms on-hand reaches target within $L + 1$ days.
<code>test_deterministic_no_demand</code>	Simulator does not place orders or accumulate costs when demand is zero.
<code>test_zero_lead_time_not_lost</code>	Goods ordered with lead time 0 arrive same-day; no units are silently dropped.
<code>test_negative_demand_clipped</code>	Negative demand values are clipped to zero; inventory does not increase from demand.
<code>test_no_negative_on_hand</code>	On-hand inventory never goes below zero across all demand and policy configurations.
<code>test_costs_and_kpis</code>	Holding, transport, ordering, and shortage costs accumulate correctly; warm-up exclusion verified.
<code>test_inventory_recovery</code>	After a demand spike, inventory recovers to base-stock within the expected number of lead-time periods.
<code>test_local_flow_conservation</code>	Units entering the network equal units leaving plus units stored (flow conservation).
<code>test_policy_behavior</code>	Policy <code>order_qty</code> returns correct values for each of the seven policy types across edge cases.
<code>test_ss_policy_behavior</code>	(s, S) -specific invariants: no order when $IP > s$; full order-up-to- S when $IP \leq s$.
<code>test_periodic_review</code>	Periodic-review policy only places orders on review days; silent on off-days.
<code>test_transport</code>	GreedyLoadPlanner vehicle packing; volume and weight dual constraints; tiered cost (full/half/quarter).
<code>test_multiroute_costs</code>	Multi-mode lane: planner selects mode that minimises cost for the given load.
<code>test_multiroute_share</code>	When multiple modes are available, load is split correctly across vehicles.
<code>test_routes_and_policies</code>	End-to-end test: correct policy applied per node/SKU combination as specified in config.
<code>test_system_properties</code>	Aggregate system-level invariants (non-negative costs, fill rate in $[0, 1]$, bullwhip ratio computed).
<code>test_finite_supplier_propagation</code>	Finite-capacity supplier correctly limits fulfilment; backlog propagates downstream.
<code>test_stochastic_leadtime</code>	Simulator handles variable lead times drawn from a distribution without losing shipments.
<code>test_112_timing</code>	Timing of receive-then-order within the daily tick for a 1-supplier, 1-warehouse, 2-retailer topology.
<code>test_variability</code>	Demand CV and order CV computed correctly; bullwhip metric is ≥ 1 for base-stock policy under variable demand.
<code>conftest</code>	68 Shared pytest fixtures: base config dict, simple 1-supplier, 1-retailer network.

APPENDIX D

REPRODUCING THE RESULTS

All results in this thesis were produced with the code in the project repository. The following commands reproduce each experiment from scratch. A Python 3.10+ virtual environment with `pip install -r requirements.txt` is assumed.

PREREQUISITES

```
# Clone repository and install dependencies  
pip install -r requirements.txt
```

RUN THE TEST SUITE

```
python -m pytest tests/ -q
```

SINGLE SIMULATION RUN (E0 BASELINE)

```
python scripts/run_simulation.py --config config/1n3_5sku.json
```

FULL EXPERIMENT SUITE (E0–E4)

```
python scripts/run_experiments.py \  
    --config config/1n3_5sku.json \  
    --trials 100 \  
    --warmup 30
```

Output is written to `outputs/experiments_<ts>/`.

RUN SPECIFIC EXPERIMENTS

```
# Run only E0 and E3
python scripts/run_experiments.py \
    --config config/1n3_5sku.json \
    --experiments E0 E3 \
    --trials 100
```

```
# Run only E4 (Pareto frontier)
python scripts/run_experiments.py \
    --config config/1n3_5sku.json \
    --experiments E4 \
    --trials 100
```

GENERATE PLOTS

```
python scripts/plot_results.py \
    --expdir outputs/experiments_<timestamp>/
```

Produces PNG plots in outputs/experiments_<ts>/plots/:
cost_breakdown.png, fill_rate.png, inventory_ts.png, orders_ts.png,
bullwhip.png, pareto_frontier.png.

MULTI-SEED STATISTICAL VALIDATION

```
python scripts/multi_seed_validation.py \
    --config config/1n3_5sku.json \
    --params outputs/experiments_<timestamp>/E3/params.json \
    --seeds 10
```

Reports 95% confidence intervals on cost and fill rate, and Welch's t -test significance for the E3 vs E0 comparison.

BUILD CONFIG FROM CSV

```
python inputs/scripts/build_config_from_csv.py
# Writes inputs/config/generated_from_csv.json

python scripts/run_simulation.py \
    --config inputs/config/generated_from_csv.json
```

EXACT RUN USED IN THESIS

The results reported in Chapter 7 are from the run directory `outputs/experiments_20260424_132309/`. The precise random seed and trial count are recorded in `outputs/experiments_20260424_132309/E*/params.json` and can be re-used to reproduce the exact results.

BIBLIOGRAPHY

1. **Akiba, T., S. Sano, T. Yanase, T. Ohta, and M. Koyama**, Optuna: A next-generation hyperparameter optimization framework. *In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2019.
2. **Arrow, K. J., T. Harris, and J. Marschak** (1951). Optimal inventory policy. *Econometrica*, **19**(3), 250–272.
3. **Bergstra, J., R. Bardenet, Y. Bengio, and B. Kégl**, Algorithms for hyper-parameter optimization. *In Advances in Neural Information Processing Systems*, volume 24. 2011.
4. **Chen, F., Z. Drezner, J. K. Ryan, and D. Simchi-Levi** (2000). Quantifying the bullwhip effect in a simple supply chain: The impact of forecasting, lead times, and information. *Management Science*, **46**(3), 436–443.
5. **Clark, A. J. and H. Scarf** (1960). Optimal policies for a multi-echelon inventory problem. *Management Science*, **6**(4), 475–490.
6. **Fu, M. C., F. W. Glover, and J. April**, Simulation optimization: A review, new developments, and applications. *In Proceedings of the 2005 Winter Simulation Conference*. 2005.
7. **Harris, F. W.** (1913). How many parts to make at once. *Factory, The Magazine of Management*, **10**(2), 135–136.
8. **Jagerman, D. L., E. M. Jewkes, and A. Mehrez** (1990). Policies for reducing inventory costs in a stochastic just-in-time environment. *European Journal of Operational Research*, **45**(2), 196–208.
9. **Lee, H. L., V. Padmanabhan, and S. Whang** (1997). Information distortion in a supply chain: The bullwhip effect. *Management Science*, **43**(4), 546–558.
10. **Makridakis, S., E. Spiliotis, and V. Assimakopoulos** (2022). The M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting*, **38**(4), 1346–1364.
11. **Pasupathy, R. and M. Poloczek** (2022). SimOpt: A testbed for simulation-optimization experiments. <https://simopt.readthedocs.io>.
12. **Roundy, R.** (1985). 98%-effective integer-ratio lot-sizing for one-warehouse multi-retailer systems. *Management Science*, **31**(11), 1416–1430.
13. **Scarf, H.** (1960). The optimality of (s, s) policies in the dynamic inventory problem. *Mathematical Methods in the Social Sciences*, 196–202.

14. **Silver, E. A.** (1978). Inventory control under a probabilistic time-varying demand pattern. *AIIE Transactions*, **10**(4), 371–379.

CURRICULUM VITAE

NAME Sakthe Balan

DATE OF BIRTH [Date of Birth]

EDUCATION QUALIFICATIONS

1970	The first degree	
	Institution	Wherever
	Specialization	Whatever

2038	The previous degree	
	Institution	Wherever
	Specialization	Whatever

Interdisciplinary Dual Degree Programme		
	Institution	Wherever
	Specialization	Whatever
	Registration Date	Whenever

COMMITTEE

Chairperson

Dr. Name
Whatever
Wherever

Guide(s)

Dr. Name
Whatever
Wherever

Dr. Name
Whatever
Wherever

Member(s)

Dr. Name
Whatever
Wherever

Dr. Name
Whatever
Wherever

Dr. Name
Whatever
Wherever